

TUGAS AKHIR - EE234899

**NAVIGASI ROBOT BERBASIS LAYERED COSTMAP
UNTUK PERGERAKAN ANTAR LANTAI PADA
LINGKUNGAN INDOOR PASCA GEMPA BUMI**

Bagas Surya Wirawan

NRP 5022221026

Dosen Pembimbing

Dr. Ir. Djoko Purwanto, M.Eng.

NIP 196512111990021002

Fajar Budiman, S.T., M.Sc.

NIP 198607072014041001

Program Studi Sarjana Teknik Elektro

Departemen Teknik Elektro

Fakultas Teknologi Elektro dan Informatika Cerdas

Institut Teknologi Sepuluh Nopember

Surabaya

2026



TUGAS AKHIR - EE234899

**NAVIGASI ROBOT BERBASIS LAYERED COSTMAP
UNTUK PERGERAKAN ANTAR LANTAI PADA
LINGKUNGAN INDOOR PASCA GEMPA BUMI**

Bagas Surya Wirawan

NRP 5022221026

Dosen Pembimbing

Dr. Ir. Djoko Purwanto, M.Eng.

NIP 196512111990021002

Fajar Budiman, S.T., M.Sc.

NIP 198607072014041001

Program Studi Sarjana Teknik Elektro

Departemen Teknik Elektro

Fakultas Teknologi Elektro dan Informatika Cerdas

Institut Teknologi Sepuluh Nopember

Surabaya

2026

[Halaman ini sengaja dikosongkan]



FINAL PROJECT - EE234899

***ROBOT NAVIGATION USING LAYERED COSTMAP FOR
INTER-FLOOR MOVEMENT IN POST-EARTHQUAKE
INDOOR ENVIRONMENTS***

Bagas Surya Wirawan

NRP 5022221026

Advisor

Dr. Ir. Djoko Purwanto, M.Eng.

NIP 196512111990021002

Fajar Budiman, S.T., M.Sc.

NIP 198607072014041001

Undergraduate Study Program of Electrical Engineering

Department of Electrical Engineering

Faculty of Intelligent Electrical and Informatics Technology

Sepuluh Nopember Institute of Technology

Surabaya

2026

[Halaman ini sengaja dikosongkan]

LEMBAR PENGESAHAN

NAVIGASI ROBOT BERBASIS LAYERED COSTMAP UNTUK PERGERAKAN ANTAR LANTAI PADA LINGKUNGAN INDOOR PASCA GEMPA BUMI

TUGAS AKHIR

Diajukan untuk memenuhi salah satu syarat
memperoleh gelar Sarjana Teknik pada
Program Studi S-1 Teknik Elektro
Departemen Teknik Elektro
Fakultas Teknologi Elektro dan Informatika Cerdas
Institut Teknologi Sepuluh Nopember

Oleh: **Bagas Surya Wirawan**
NRP. 5022221026

Disetujui oleh Tim Penguji Tugas Akhir:

Dr. Ir. Djoko Purwanto, M.Eng.
NIP: 196512111990021002

(Pembimbing I)

.....

Fajar Budiman, S.T., M.Sc.
NIP: 198607072014041001

(Pembimbing II)

.....

Ir. Tasripan, M.T..
NIP: 196204181990031004

(Penguji I)

.....

Ir. Harris Pirngadi, M.T..
NIP: 196205101989031001

(Penguji II)

.....

SURABAYA
Juli, 2026

[Halaman ini sengaja dikosongkan]

PERNYATAAN ORISINALITAS

Yang bertanda tangan dibawah ini:

Nama Mahasiswa / NRP : Bagus Surya Wirawan / 5022221026
Departemen : Teknik Elektro
Dosen Pembimbing 1/ NIP : Dr. Ir. Djoko Purwanto, M.Eng. / 196512111990021002
Dosen Pembimbing 2/ NIP : Fajar Budiman, S.T., M.Sc. / 198607072014041001

Dengan ini menyatakan bahwa Tugas Akhir dengan judul "NAVIGASI ROBOT BERBASIS LAYERED COSTMAP UNTUK PERGERAKAN ANTAR LANTAI PADA LINGKUNGAN INDOOR PASCA GEMPA BUMI" adalah hasil karya sendiri, berfsifat orisinal, dan ditulis dengan mengikuti kaidah penulisan ilmiah.

Bilamana di kemudian hari ditemukan ketidaksesuaian dengan pernyataan ini, maka saya bersedia menerima sanksi sesuai dengan ketentuan yang berlaku di Institut Teknologi Sepuluh Nopember.

Surabaya,..... 2026

Mengetahui
Dosen Pembimbing 1

Dosen Pembimbing 2

Dr. Ir. Djoko Purwanto, M.Eng.
NIP. 196512111990021002

Fajar Budiman, S.T., M.Sc.
NIP. 198607072014041001

Mahasiswa

Bagas Surya Wirawan
NRP. 5022221026

[Halaman ini sengaja dikosongkan]

ABSTRAK

Nama Mahasiswa : Bagas Surya Wirawan
Judul Tugas Akhir : NAVIGASI ROBOT BERBASIS LAYERED COSTMAP UNTUK PERGERAKAN ANTAR LANTAI PADA LINGKUNGAN INDOOR PASCA GEMPA BUMI
Pembimbing : 1. Dr. Ir. Djoko Purwanto, M.Eng.
2. Fajar Budiman, S.T., M.Sc.

Indonesia mengalami frekuensi gempa bumi tinggi yang menyebabkan perubahan struktural signifikan pada lingkungan *indoor* bertingkat. Robot otonom dapat berperan menavigasi area *indoor* pasca gempa yang tidak aman bagi manusia. Untuk melakukan misi tersebut, robot memerlukan kemampuan navigasi yang mampu bekerja pada lingkungan pasca gempa yang bertingkat. Tugas akhir ini membahas pengembangan metode navigasi menggunakan *layered costmap* supaya robot dapat melintasi lingkungan tersebut. Skema ini merepresentasikan lingkungan bertingkat sebagai kumpulan peta *region* berbentuk *occupancy grid* yang digabung menjadi satu *map*. Dalam *costmap* berlapis, tiap lapisan pada *costmap* menyimpan informasi berbeda seperti rintangan statis, rintangan dinamis, lapisan inflasi, dan lapisan transisi antar *map*, yang memungkinkan navigasi antar *region*. Dalam arsitektur navigasi yang dikembangkan, setiap lantai pada lingkungan *indoor* direpresentasikan oleh satu *region* individual. Pendekatan ini menyederhanakan informasi lingkungan 3D menjadi representasi 2D yang dapat dikelola, sehingga memungkinkan penggunaan sensor *LiDAR* 2D untuk menentukan lingkungan robot. Algoritma navigasi kemudian mengintegrasikan *map* 2D tiap *region* ini menjadi satu kesatuan dengan menggunakan *costmap* transisi, sehingga memungkinkan robot untuk memahami dan menavigasi lingkungan multi-lantai. Sistem ini memungkinkan robot untuk menavigasi dari satu *region* ke *region* lain dengan (1) estimasi pose menggunakan *iterative closest point* (ICP) dan *region switcher*, (2) *path planning* yang membuat rute antar *region* menggunakan *path router*, dan (3) *path tracking* yang menggunakan *dynamic window approach* (DWA) *planner* untuk menghasilkan *trajectory* yang diikuti robot. Dari hasil pengujian pada simulator MuJoCo, robot berhasil melakukan navigasi dari *region* A sampai C dengan tingkat keberhasilan 80% (8 dari 10 percobaan), waktu tempuh rata-rata 199.35 detik, dan waktu terbaik 141.01 detik.

Kata Kunci: *Navigasi, Costmap, Robot, Antar lantai*

[Halaman ini sengaja dikosongkan]

ABSTRACT

Name : Bagas Surya Wirawan
Title : *ROBOT NAVIGATION USING LAYERED COSTMAP FOR INTER-FLOOR MOVEMENT IN POST-EARTHQUAKE INDOOR ENVIRONMENTS*
Advisors : 1. Dr. Ir. Djoko Purwanto, M.Eng.
2. Fajar Budiman, S.T., M.Sc.

Indonesia experiences a high frequency of earthquakes, causing significant structural changes to multi-story indoor environments. Autonomous robots can play a role in navigating post-earthquake indoor areas that are unsafe for humans. To accomplish this mission, robots require navigation capabilities that can operate in multi-story post-earthquake environments. This final project discusses the development of a navigation method using a layered costmap to enable robots to traverse such environments. This scheme represents a multi-story environment as a collection of region maps in the form of occupancy grids merged into a single map. In a layered costmap, each layer stores different types of information, such as static obstacles, dynamic obstacles, inflation layer, and transition layers between maps, enabling inter-region navigation. In the developed navigation architecture, each floor of an indoor environment is represented by an individual region. This approach simplifies 3D environmental information into a manageable 2D representation, allowing the use of a 2D LiDAR sensor to perceive the robot's surroundings. The navigation algorithm then integrates the 2D maps of each region into a unified system using a transition costmap, enabling the robot to understand and navigate multi-story environments. The system allows the robot to navigate from one region to another through (1) pose estimation using iterative closest point (ICP) and a region switcher, (2) path planning that generates inter-region routes using a path router, and (3) path tracking that uses a dynamic window approach (DWA) planner to generate the trajectory followed by the robot. From the simulation results on the MuJoCo simulator, the robot successfully navigated from region A to C with an 80% success rate (8 out of 10 trials), an average travel time of 199.35 seconds, and a best time of 141.01 seconds.

Keywords: Navigation, Costmap, Robot, Inter-floor.

[Halaman ini sengaja dikosongkan]

KATA PENGANTAR

Penulis mengucapkan syukur kepada Tuhan Yang Maha Esa atas limpahan rahmat dan karunia-Nya sehingga tugas akhir berjudul "NAVIGASI ROBOT BERBASIS LAYERED COSTMAP UNTUK PERGERAKAN ANTAR LANTAI PADA LINGKUNGAN INDOOR PASCA GEMPA BUMI" ini dapat diselesaikan dengan baik dan selesai tepat waktu.

Penulis menyadari bahwa terselesaikannya tugas akhir ini tidak lepas dari kontribusi berbagai pihak, baik secara nyata atau emosional. Untuk itu, penulis dengan tulus menyampaikan terima kasih kepada:

1. Tuhan Yang Maha Esa atas segala rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan tugas akhir ini.
2. Kedua orang tua beserta seluruh keluarga yang selalu memberikan dukungan doa dan emosional kepada penulis sehingga penulis dapat kuat menyelesaikan tugas akhir ini.
3. Bapak Dr. Ir. Djoko Purwanto, M.Eng. dan Fajar Budiman, S.T., M.Sc. selaku dosen pembimbing yang telah memberikan bimbingan, arahan, serta motivasi kepada penulis selama penyusunan tugas akhir ini.
4. Zamzamy, Faizzul, Rayhan, Danial, Figo, Nasywa, Wayan, Hafizh, Ikhsan, Bagas Abimanyu, Zahran, Rasya, Yuan, Quratta, Dipta, dan seluruh teman-teman dari tim Abinara-1 yang telah banyak membantu penulis selama penyusunan tugas akhir ini.
5. Seluruh dosen dan tenaga kependidikan di Departemen Teknik Elektro ITS yang telah memberikan ilmu pengetahuan dan pelayanan yang baik selama penulis menempuh pendidikan di Departemen Teknik Elektro.
6. Teman-teman sesama pejuang Tugas Akhir Departemen Teknik Elektro ITS angkatan 2022 yang senantiasa memberikan dukungan, perhatian, dan motivasi
7. Seluruh pihak yang tidak dapat penulis sebutkan satu per satu yang telah membantu penulis dalam menyelesaikan tugas akhir ini.

Penulis menyadari bahwa dalam penyusunan tugas akhir ini masih banyak kekurangan dan jauh dari kesempurnaan. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun demi kesempurnaan tugas akhir ini. Semoga tugas akhir ini dapat memberikan manfaat bagi semua pihak yang membacanya.

Surabaya, Juli 2026

Bagas Surya Wirawan

[Halaman ini sengaja dikosongkan]

DAFTAR ISI

ABSTRAK	i
ABSTRACT	iii
KATA PENGANTAR	v
DAFTAR ISI	vii
DAFTAR GAMBAR	xi
DAFTAR TABEL	xiii
1 PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	2
1.3 Batasan Masalah	2
1.4 Tujuan	2
1.5 Manfaat	3
2 TINJAUAN PUSTAKA	5
2.1 Hasil Penelitian Terdahulu	5
2.2 Teori Dasar	6
2.2.1 Occupancy Grid	6
2.2.2 Layered Costmap	6
2.2.3 Lokalisasi Iterative Closest Point (ICP)	6
2.2.4 Path Planning A*	8
2.2.5 Trajectory Generator	8
2.2.6 ROS	9
3 METODOLOGI	11
3.1 Bahan dan Peralatan	11
3.1.1 Model robot hexapod	11
3.1.2 Simulasi Sensor	11
3.1.3 Perangkat Keras	12

3.1.4	Perangkat lunak pendukung	13
3.2	Pengolahan Lingkungan 3D Menjadi Map 2D	13
3.2.1	Map Region	14
3.2.2	Map Transisi	15
3.2.3	Representasi Topologi Graph	16
3.3	Perancangan Sistem Navigasi	18
3.3.1	Lokalisasi Iterative Closest Point (ICP)	19
3.3.2	Costmap	23
3.3.3	Region Switcher	29
3.3.4	Global Planner	31
3.3.5	Path Router	34
3.3.6	Dynamic Window Approach (DWA) Local Planner	35
3.4	Metode Pengujian dan Evaluasi	39
3.4.1	Definisi Kondisi Pasca Gempa	39
3.4.2	Pengujian Lokalisasi ICP	39
3.4.3	Pengujian Global Planner (A*)	40
3.4.4	Pengujian Local Planner (DWA)	41
3.5	Urutan Pelaksanaan Penelitian	41
4	HASIL DAN PEMBAHASAN	45
4.1	Hasil dan Analisis Lokalisasi ICP	45
4.1.1	Hasil Zona Transisi	46
4.2	Hasil dan Analisis Global Planner	47
4.2.1	Hasil Path di Region A	47
4.2.2	Hasil Path di Region B	48
4.2.3	Hasil Path di Region C	49
4.2.4	Hasil Deviasi Path	50
4.3	Hasil dan Analisis DWA Local Planner	52
4.4	Pembahasan	52
4.4.1	Pembahasan Costmap	52
4.4.2	Pembahasan Lokalisasi	53
4.4.3	Pembahasan Path Planner	53
4.4.4	Pembahasan DWA Local Planner	54
4.4.5	Pembahasan Navigasi Penuh dari Region A ke C	54

5	PENUTUP	57
5.1	Kesimpulan	57
5.2	Saran	57
	DAFTAR PUSTAKA	59
	LAMPIRAN	61
	BIOGRAFI PENULIS	65

[Halaman ini sengaja dikosongkan]

DAFTAR GAMBAR

3.1	Model platform navigasi yang digunakan dalam simulasi	11
3.2	Referensi sensor yang digunakan dalam simulasi (Shenzhen, 2024)	12
3.3	Diagram perangkat keras pada robot asli	12
3.4	Lingkungan arena dalam 3D	14
3.5	Map Occupancy Grid masing-masing region	14
3.6	Map Gabungan	15
3.7	Map transisi antar region	15
3.8	Representasi graph dari map transisi	17
3.9	Arsitektur sistem navigasi	18
3.10	Diagram alur navigasi antar lantai	19
3.11	Diagram alir proses ICP	19
3.12	Visualisasi grid masing-masing costmap	24
3.13	Visualisasi <i>global costmap</i> pada RViz	27
3.14	Diagram alir region switcher	29
3.15	Diagram alir proses global planner	31
3.16	Diagram alir proses path router	34
3.17	Diagram alir proses DWA local planner	35
3.18	Elemen kondisi pasca gempa	39
3.19	Visualisasi lingkungan simulasi di berbagai region	40
4.1	Perbandingan error ICP di Region A	45
4.2	Error ICP di Region C	46
4.3	Visualisasi error ICP dan radius zona transisi	46
4.4	Relasi path terhadap Cost Scaling Factor di Region A	47
4.5	Semua path di region A.	47
4.6	Relasi path terhadap Cost Scaling Factor di Region B	48
4.7	Semua path di region B.	49
4.8	Relasi path terhadap Cost Scaling Factor di Region C	49
4.9	Semua path di region C.	50

[Halaman ini sengaja dikosongkan]

DAFTAR TABEL

3.1	Daftar Koneksi pasangan	16
3.2	Orientasi Region terhadap Global Frame	16
3.3	Posisi Inisial Robot	16
3.4	Daftar node pada topological graph	17
3.5	Nilai konstanta costmap	23
3.6	Komposisi layer pada masing-masing costmap	24
3.7	Daftar kriteria penilaian lintasan	37
3.8	Parameter ICP	40
3.9	Parameter Default Cost Function	40
3.10	Konfigurasi parameter DWA setiap region	41
3.11	Tabel timeline	43
4.1	Karakteristik Path di Region A berdasarkan Cost Scaling Factor	47
4.2	Karakteristik Path di Region B berdasarkan Cost Scaling Factor	48
4.3	Karakteristik Path di Region C berdasarkan Cost Scaling Factor	50
4.4	Perbandingan path terhadap ground truth di region A	50
4.5	Perbandingan path terhadap ground truth di region B	51
4.6	Perbandingan path terhadap ground truth di region C	51
4.7	Rangkuman cross track error per run	52
4.8	Rangkuman waktu tempuh	54

[Halaman ini sengaja dikosongkan]

BAB I

PENDAHULUAN

1.1 Latar Belakang

Indonesia adalah negara yang sering sekali mengalami bencana gempa bumi. Secara geografis, Indonesia terletak di antara 4 lempeng tektonik, yaitu Lempeng Pasifik, Lempeng Eurasia, Lempeng Indo-Australia, dan Lempeng Laut Filipina. Lokasi Indonesia di antara 4 lempeng ini menyebabkan frekuensi terjadinya gempa bumi cukup tinggi. Berdasarkan data dari BMKG, rata-rata Indonesia mengalami 18 gempa bumi tiap hari (Sabtaji, 2020). Pasca gempa bumi, operasi *search and rescue* (SAR) akan dilakukan di daerah yang terdampak gempa bumi untuk menolong korban dan menanggulangi kerusakan. Seringkali petugas SAR mengalami ancaman terhadap keselamatannya dalam melaksanakan operasi SAR.

Robot bisa ikut berperan dalam membantu petugas SAR melaksanakan tugasnya. Robot UAV mampu memberi gambaran medan pada daerah terjadinya bencana, sedangkan robot darat yang *track* atau *quadruped* dapat membantu membawa barang berat dan membantu dalam hal lainnya pada saat area bencana tidak dapat dilewati kendaraan besar. Robot juga dapat melakukan misi yang terlalu berbahaya untuk dilakukan oleh manusia. Sebagai contoh, sebuah robot darat pernah digunakan pasca gempa di Jepang untuk menginspeksi bangunan yang mengalami kerusakan. Mengirim petugas manusia dinilai terlalu berbahaya karena atap bangunan yang hampir roboh, sehingga digunakan robot yang dikendalikan dari jarak jauh (Lin et al., 2022).

Dalam implementasinya, robot darat SAR dapat dikendalikan manual oleh operator atau bisa otonom. Walaupun robot yang dikendalikan operator memiliki algoritma yang lebih sederhana, robot otonom juga memiliki peran pada operasi SAR, terutama pada daerah di mana infrastruktur komunikasinya rusak (Zhang et al., 2023). Untuk bisa melakukan misi otonom di dalam bangunan, robot darat perlu bisa melakukan lokalisasi, menemukan jalur navigasi yang aman, dan mencapai posisi tujuan, pada lingkungan tidak terstruktur, seperti pada lingkungan indoor dengan banyak puing yang disebabkan oleh gempa. Meskipun beberapa robot mampu bernavigasi pada lingkungan tidak terstruktur, navigasi robot darat pada area indoor masih menjadi permasalahan untuk robot otonom.

Dalam proses pembuatan algoritma, sulit untuk diuji pada kondisi pasca gempa yang sebenarnya, sehingga diuji pada lingkungan yang mensimulasikan kondisi gempa. Lingkungan simulasi diadaptasi dari arena yang digunakan untuk Kontes Robot SAR Indonesia (KRSRI) 2024 (Kusumoputro et al., 2023). Platform yang digunakan adalah robot hexapod yang dilengkapi dengan sensor LiDAR 2D. Keterbatasan LiDAR 2D pada lingkungan pasca gempa yang bertingkat merupakan tantangan tersendiri dalam implementasi sistem navigasi.

Di dalam tugas akhir ini, diusulkan sebuah metode untuk mencoba menjawab permasalahan tersebut. Pada implementasi navigasi untuk robot darat otonom, umum digunakan *occupancy grid* dan *costmap* 2D untuk menyimpan informasi lingkungan. Metode ini terbatas saat robot perlu mencapai lokasi yang tidak terletak pada bidang yang sama. Untuk membenahi ini, sebuah metode untuk menyederhanakan informasi lingkungan 3D menjadi direpresentasikan oleh beberapa bidang *costmap* 2D atau *layered costmap*. Tiap *map* dapat mewakili sebuah lan-

tai di dalam bangunan atau lingkungan indoor. Lalu, robot darat otonom dapat bergerak pada salah satu *map* tertentu, kemudian berpindah ke *map* lain sesuai posisinya di bangunan.

Metode *layered costmap* yang diusulkan dalam tugas akhir telah digunakan sebagai bagian dari sistem navigasi robot otonom Pada penelitian-penelitian terdahulu. Perbedaan disini terletak pada penambahan *layer* baru yang digunakan pada navigasi robot di lingkungan bertingkat, dimana belum dieksplorasi pada penelitian sebelumnya. Selain itu, navigasi antar lantai juga pernah dilakukan pada studi terdahulu namun, penelitian tersebut umumnya memisahkan peta tiap lantai pada file masing-masing. Berbeda dengan pendekatan tersebut, tugas akhir ini mengajukan metode dengan menggunakan satu peta gabungan (*combined map*) yang menyatukan seluruh region ke dalam satu representasi *costmap* 2D.

1.2 Rumusan Masalah

Berdasarkan latar belakang pada bagian sebelumnya, terdapat beberapa inti masalah yang perlu dibahas. Permasalahan tersebut dijabarkan sebagai berikut:

1. Bagaimana cara menggunakan *layered costmap* untuk mewakili informasi lingkungan 3D yang terdiri dari beberapa lantai sehingga bisa digunakan pada algoritma navigasi 2D?
2. Bagaimana cara membuat algoritma lokalisasi, *path planning*, dan *path tracking* yang bisa bekerja pada algoritma navigasi yang menggunakan *layered costmap*?
3. Apakah dengan menggunakan metode navigasi antar lantai, robot darat otonom dapat melakukan navigasi hingga mencapai tujuan pada lingkungan simulasi?

1.3 Batasan Masalah

Dalam tugas akhir ini, metode multilayer *costmap* yang dikembangkan tidak dirancang untuk langsung digunakan di kondisi pasca gempa. Hal ini disebabkan oleh beberapa faktor berikut:

1. Pengujian dilakukan pada simulasi yang didesain untuk merekayasa kondisi pasca gempa. Arena dilengkapi dengan rekayasa rintangan tidak diketahui dan permukaan tidak rata, yang dibuat pada skala lebih kecil.
2. Navigasi dilakukan oleh robot darat hexapod yang melakukan navigasi secara otonom, dengan bergerak dari titik awal menuju tujuan yang telah ditentukan.
3. Algoritma navigasi digunakan pada kondisi sudah diketahui lingkungan utamanya (*known map*).

1.4 Tujuan

Tugas akhir ini bertujuan untuk:

1. Mengembangkan metode representasi lingkungan menggunakan *layered costmap* untuk menyederhanakan informasi lingkungan yang terdiri dari beberapa lantai, yang dapat digunakan oleh robot otonom untuk navigasi antar lantai.
2. Merancang dan mengimplementasikan algoritma lokalisasi, *path planning*, dan *path tracking*, yang menggunakan *layered costmap* agar robot dapat bernavigasi antar lantai.
3. Melakukan pengujian sistem navigasi berbasis *layered costmap* di lingkungan simulasi, untuk mengevaluasi efektivitas dan keterbatasan dari pendekatan yang diusulkan sebelum diterapkan pada skenario nyata.

1.5 Manfaat

Tugas akhir ini diharapkan dapat memberikan manfaat sebagai berikut:

1. Penelitian ini memberikan kontribusi terhadap pengembangan sistem navigasi robot otonom, khususnya dalam pemrosesan data lingkungan kompleks menjadi representasi yang lebih sederhana dan terstruktur.
2. Metode *layered costmap* yang diusulkan dapat menjadi dasar awal untuk navigasi robot di area bertingkat atau tidak rata, seperti bangunan runtuh pasca gempa.
3. Dasar untuk penelitian lanjutan dalam pengembangan sistem robot yang mampu beroperasi secara otonom di area terdampak bencana, dengan mempertimbangkan faktor medan nyata yang kompleks dan tidak terstruktur

[Halaman ini sengaja dikosongkan]

BAB II

TINJAUAN PUSTAKA

2.1 Hasil Penelitian Terdahulu

Penelitian oleh (Macenski et al., 2023) membahas implementasi *layered costmap* di dalam library ROS 2, yaitu sebuah metode untuk menyimpan informasi lingkungan secara sistematis dengan tiap lapisan mampu menyimpan informasi tertentu. Tiap lapisan dapat mewakili sumber informasi, algoritma, ataupun hasil berbeda. Standarnya di ROS 2 *layered costmap* dibagi menjadi lapisan static, inflation, obstacle, dan voxel. Manfaatnya dengan menggunakan *layered costmap* termasuk pembaruan yang lebih jelas, area pembaruan yang dinamis, proses pembaruan yang teratur, dan konfigurasi yang fleksibel, memungkinkan representasi nilai cost yang kompleks dan perilaku navigasi yang membutuhkan konteks, seperti navigasi sosial.

Studi oleh (Hu et al., 2022) mengusulkan pendekatan navigasi adaptif kemiringan untuk robot bergerak darat, yang bertujuan mengatasi keterbatasan costmap 2-dimensi yang sering salah mengartikan kemiringan (slope) sebagai area terlarang. Pendekatan baru ini didasarkan pada *layered costmap* yang secara aktif mengintegrasikan informasi kemiringan selama tahap pembuatan peta lingkungan. Sebuah algoritma deteksi kemiringan dikembangkan untuk mengidentifikasi kemiringan dan secara adaptif mengubah peta biaya selama navigasi, sehingga kemiringan dianggap sebagai jalur yang dapat dilalui, hanya dengan biaya tambahan.

Beberapa penelitian sebelumnya mengusulkan metode untuk mewakili informasi multi-lantai dari bangunan. Studi oleh (T. Kim et al., 2024) menguraikan pengembangan *indoor delivery mobile robot* dengan arsitektur yang dirancang khusus untuk lingkungan multi-lantai, dengan menggunakan peta navigasi terintegrasi yang dibuat dengan menggabungkan peta *point cloud* multi-lantai dan peta topologi berbasis node graph. Studi oleh (Palacín, Bitriá, et al., 2023) membuat metode navigasi antar lantai di gedung bertingkat dengan menggunakan lift, dengan lokalisasi algoritma Iterative Closest Point (ICP). Performa ICP dievaluasi dalam berbagai kondisi, yaitu saat pintu lift dalam proses terbuka dan tertutup. Sedangkan, (Jung et al., 2024) menggunakan sensor barometer untuk mendeteksi perbedaan ketinggian antar lantai saat menggunakan lift.

Beberapa penelitian membuat metode untuk secara otomatis mendeteksi perantara pergantian lantai seperti lift dan tangga. Paper oleh (Li et al., 2021) menunjukan robot melakukan navigasi gedung bertingkat dengan mengoperasikan lift secara otonom. Sedangkan, penelitian oleh (Lee et al., 2023) dan (E.-H. Kim et al., 2020) memanfaatkan pengenalan tombol lift dengan *deep learning* untuk memungkinkan robot beroperasi secara mandiri di antara lantai tanpa memerlukan modifikasi fasilitas lift yang ada atau sistem manajemen gedung tambahan. Penelitian oleh (Zhu et al., 2020) mengusulkan metode *real-time* untuk mendeteksi dan lokalisasi tangga dengan sensor LiDAR VL-16P yang digunakan untuk mendeteksi fitur geometris dari tangga

Untuk metode *path planning*, (Xie & Zhou, 2023) membuat sistem navigasi untuk melintasi lantai dan tangga dengan mengintegrasikan algoritma A* untuk perencanaan jalur awal dan *DWA-based path following* untuk permukaan datar. Paper oleh (Jung et al., 2024) men-

gusulkan skema untuk perencanaan lintasan menggunakan algoritma A* yang ditambah dengan fungsi biaya untuk mempertimbangkan faktor-faktor realistis seperti waktu tunggu lift, dan ini memungkinkan robot memilih mode pergerakan antar lantai (lift atau tangga). Penelitian oleh (Palacín, Rubies, et al., 2023) menggunakan *static navigation tree* yang menggambarkan *weighted paths* dalam bangunan bertingkat. Lalu, *navigation tree* digunakan untuk merencanakan jalur dan mengestimasi durasi dari semua rute pengiriman menggunakan algoritma dijkstra.

2.2 Teori Dasar

2.2.1 Occupancy Grid

Occupancy Grid adalah salah satu metode pemetaan spasial yang paling fundamental dalam robotika, di mana ruang lingkungan direpresentasikan sebagai sebuah kisi (grid) yang terdiri dari sel-sel berukuran sama. Sejauh ini, domain yang paling umum dari peta *occupancy grid* (kisi keterhunian) adalah peta denah lantai 2-D, yang menggambarkan irisan 2-D dari dunia 3-D. Peta 2-D seringkali sudah cukup, terutama ketika robot bernavigasi di atas permukaan datar dan sensor dipasang sedemikian rupa sehingga hanya menangkap satu irisan dari dunia tersebut. Teknik *occupancy grid* dapat digeneralisasi menjadi representasi 3-D, tetapi membutuhkan beban komputasi yang signifikan.

Misalkan $\mathbf{m}_i$ menyatakan sel kisi dengan indeks i . Sebuah peta *occupancy grid* mempartisi ruang menjadi sejumlah sel kisi yang terbatas:

$$m = \sum_i \mathbf{m}_i \quad (9.2)$$

Setiap $\mathbf{m}_i$ memiliki nilai keterhunian biner yang terikat padanya, yang menentukan apakah sebuah sel terisi (*occupied*) atau kosong (*free*). Kita akan menuliskan '1' untuk terisi dan '0' untuk kosong (Thrun, 2002).

2.2.2 Layered Costmap

Costmap adalah representasi grid dua dimensi dari lingkungan robot yang digunakan dalam sistem navigasi, dengan tiap sel pada grid memiliki harga yang terasosiasi dengannya. Pada *costmap* modern, tiap sel merupakan integer dari 0 hingga 255 yang merepresentasikan biaya navigasi. Secara umum, algoritma perencanaan jalur akan mengutamakan melewati sel dengan harga rendah dan menghindari sel dengan harga tinggi. Metode *costmap* memberikan keseimbangan antara kualitas informasi dan efisiensi algoritma. Sebagai ekstensi dari *costmap*, digunakan konsep *layered costmap*, yaitu struktur costmap yang terbagi menjadi beberapa layer terpisah berdasarkan jenis data atau fungsi spesifik (misalnya static map, obstacle, inflation, proxemic). Setiap layer memodifikasi master costmap secara terstruktur dan independen, memungkinkan sistem navigasi mempertimbangkan konteks yang lebih kompleks dan menghasilkan perilaku pergerakan robot yang lebih cerdas dan adaptif terhadap lingkungan (Macenski et al., 2023).

2.2.3 Lokalisasi Iterative Closest Point (ICP)

Lokalisasi robot adalah proses untuk menentukan posisi dan orientasi robot (pose) relatif terhadap peta lingkungan tempat robot beroperasi. Kemampuan ini sangat penting bagi robot

untuk melakukan navigasi dan pengambilan keputusan yang tepat dalam menjalankan misi otonom. Lokalisasi umumnya menggunakan informasi dari sensor internal, seperti encoder roda dan IMU, dan sensor eksternal, seperti kamera atau LiDAR, untuk memperkirakan posisi robot berdasarkan model gerak dan observasi lingkungan.

Salah satu kategori metode lokalisasi adalah map-matching, dimana data sensor dari lingkungan dicoba untuk disamakan dengan fixed map yang sudah ada. Algoritma iterative closest point (ICP) merupakan salah satu algoritma yang bekerja dengan metode map-matching ini. Cara kerja algoritma ICP adalah dengan mencoba mengurangi jarak Euclidean antara input data dengan sebuah model referensi (dalam lokalisasi adalah fixed map). Secara matematis, bayangkan terdapat dua set titik 2D A dan B. Tujuan algoritma adalah menemukan sebuah transformasi A ke B untuk mencari mean squared distance (MSD) antara titik A dan B (Sobreira et al., 2019).

$$MSD(A, B, u) = \frac{1}{n} \sum_{a \in A, b \in B} \|b - u(a)\|^2 \quad (2.1)$$

Dengan rumus matematika diatas, algoritma ICP mencoba untuk mengurangi MSD pada setiap iterasi. ICP terbagi menjadi dua tahap, yaitu matching stage dan transformation stage. Matching stage adalah tahap pertama dari ICP dan bertujuan untuk meminimalkan nilai mean square distance dengan dengan matching terbaik antara set titik A dan B.

Transformation stage adalah tahap kedua yang bertujuan untuk mencari nilai paling optimal dari transformasi rotasi R dan translasi t untuk mencapai MSD paling kecil. ICP menggunakan simple least square solver untuk menemukan transformasi matrix (R — t) yang meminimalkan MSD. untuk mencapai hal tersebut dilakukan pengurangan dari data referensi dan point cloud sensor nilai centroid masing-masing, yang ditunjukkan di rumus dibawah (Sobreira et al., 2019).

$$a'_i = a_i - \frac{1}{n} \sum_{i=0}^n a_i \quad (2.2)$$

$$b'_i = b_i - \frac{1}{m} \sum_{i=0}^m b_i \quad (2.3)$$

Setelah itu, nilai kovarian silang matrix dikomputasi menggunakan persamaan dibawah dengan A' dan B' sama dengan set poin a'_i dan b'_i.

$$H = A' B'^T \quad (2.4)$$

Lalu, sudut rotasi θ bisa dihitung dengan rumus dibawah .

$$\theta = \text{atan2}((H(0, 1) - H(1, 0)), (H(0, 0) + H(1, 1))) \quad (2.5)$$

Pada akhir tahap transformasi, data sensor diubah menggunakan matriks (R — t) yang diperkirakan dan algoritma kembali ke tahap pencocokan, kecuali jika kriteria penghentian diverifikasi (misalnya, jumlah iterasi, peningkatan kesalahan Euclidean antar iterasi, dll.) (So-

breira et al., 2019).

$$R = m \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix} \quad (2.6)$$

$$t = \bar{b} - R\bar{a} \quad (2.7)$$

2.2.4 Path Planning A*

Path planning adalah proses perencanaan jalur geometris yang menghubungkan titik awal ke titik tujuan tanpa menabrak rintangan, biasanya dilakukan dalam ruang konfigurasi (configuration space) dengan mempertimbangkan berbagai batasan lingkungan. Metode *path planning* umum mencakup teknik roadmap, cell decomposition, dan artificial potential field.

Salah satu algoritma *path planning* yang paling sering digunakan adalah A*. A* dikategorikan sebagai algoritma best first search (BFS) yang menggabungkan kelebihan uniform-cost dan greedy searches dengan menggunakan fitness function.

$$f(n) = g(n) + h(n) \quad (2.8)$$

Pada persamaan diatas, $g(n)$ adalah biaya kumulatif dari titik awal ke simpul n , dan $h(n)$ adalah estimasi heuristik biaya tersisa menuju simpul tujuan. Selama pencarian, A* mengelola daftar terbuka (simpul yang akan dipertimbangkan) dan daftar tertutup (simpul yang sudah dikunjungi). Algoritma ini bekerja dengan memperluas simpul dari daftar terbuka dengan fungsi kecocokan minimal, memindahkannya ke daftar tertutup, dan menambahkan tetangganya ke daftar terbuka hingga simpul tujuan diperluas.

Pilihan heuristik yang baik sangat penting untuk kualitas dan efisiensi pencarian A*. Heuristik yang mengestimasi biaya nyata menjamin jalur terpendek, tetapi bisa memperluas terlalu banyak simpul. Sebaliknya, heuristik yang melebih-lebihkan dapat mempercepat pencarian ke tujuan, namun berisiko melambatkan jika jalur optimal menyimpang. Meskipun A* banyak digunakan karena menemukan jalur biaya minimum dan memastikan keberadaan jalur bebas, kelemahan utamanya adalah konsumsi waktu yang tinggi dan berat komputasi yang signifikan (Damle & Susan, 2022).

2.2.5 Trajectory Generator

Pembuatan trajectory atau *Path tracking* bertugas menambahkan informasi waktu pada jalur tersebut, sehingga menghasilkan lintasan yang dapat diikuti oleh robot secara fisik. Proses ini mempertimbangkan batasan kinematik dan dinamik dari robot, termasuk kecepatan, percepatan, dan jerk, untuk menghasilkan lintasan yang halus, aman, dan efisien. *path tracking* memastikan bahwa gerakan aktual robot mengikuti jalur yang telah direncanakan dengan akurat dan memastikan kestabilan robot serta performa yang sesuai dengan sistem kontrol yang digunakan (Yao et al., 2020).

Untuk membuat gerakan robot yang mulus (tidak menyentak atau jerky), kita tidak bisa sekadar memindahkan kecepatan dari 0 ke 1 secara mendadak. Dua metode trajectory yang paling umum digunakan:

1. **Polynomial (Cubic dan Quintic):** Fungsi waktu dibuat menggunakan persamaan polinomial. Polinomial orde-3 (Cubic) digunakan untuk memastikan kecepatan robot di titik awal dan titik akhir bernilai nol. Polinomial orde-5 (Quintic) selangkah lebih maju den-

gan memastikan percepatannya juga nol di titik awal dan akhir, sehingga pergerakan jauh lebih halus bagi motor robot.

2. **Profil Kecepatan Trapezium:** Robot berakselerasi dengan nilai konstan di awal, melaju dengan kecepatan konstan di pertengahan jalur, lalu melambat dengan deselerasi konstan di akhir. Jika digrafikkan terhadap waktu, kecepatannya akan membentuk bangun trapesium. Ini adalah metode yang paling banyak digunakan di industri.

Dalam praktiknya, robot jarang hanya bergerak dari titik A ke titik B. Robot sering harus melewati banyak titik (waypoints). Jika robot harus benar-benar berhenti di setiap titik, gerakannya akan sangat kaku dan memakan waktu. Daripada berhenti total, lintasan robot dikalkulasi agar bisa "memotong sudut" di dekat waypoint tanpa mengurangi kecepatan hingga nol, menjaga momentum dan fluiditas pergerakan (Lynch & Park, 2017).

2.2.6 ROS

Robot Operating System (ROS) merupakan sebuah kerangka kerja middleware opensource yang menyediakan navigation stack komprehensif dengan mengintegrasikan fungsi persepsi lingkungan, konstruksi peta, perencanaan jalur, dan kontrol gerakan. Sistem pada ROS beroperasi menggunakan arsitektur modular yang memisahkan antara perangkat keras dan perangkat lunak. Modul-modul fungsional di dalam ROS dihubungkan melalui mekanisme komunikasi publish-subscribe, yang memungkinkan terjadinya pemisahan modul secara fleksibel, penggantian plugin, dan penyesuaian parameter secara langsung saat sistem sedang berjalan. Arsitektur closed-loop ini menyediakan fondasi yang sangat tangguh dan adaptif untuk mendukung operasional navigasi otonom pada robot seluler.

Di dalam arsitektur navigasi ROS, lapisan path planning berperan penting dalam mengarahkan pergerakan robot dengan aman, yang pada umumnya terdiri dari dua komponen utama, perencanaan jalur global (seperti A* dan Dijkstra) serta penghindaran rintangan lokal (seperti DWA dan TEB). Algoritma-algoritma pada lapisan ini secara dinamis menghasilkan lintasan yang optimal berdasarkan pemodelan peta saat ini serta informasi rintangan yang ada di sekitarnya. Di samping itu, lapisan perencanaan ini juga dirancang untuk mendukung kemampuan perencanaan ulang secara waktu nyata, sehingga robot dapat merespons dan menghindari rintangan dinamis secara efektif di dalam lingkungan yang kompleks.

Perencana jalur global (global planner) bertugas untuk mengkalkulasi rute keseluruhan secara makro pada peta statis, di mana algoritma A* dan Dijkstra adalah metode yang paling luas penggunaannya. Untuk mengeksekusi rute global sekaligus menghindari rintangan lokal secara langsung, sistem ROS umumnya memanfaatkan Dynamic Window Approach (DWA) sebagai plugin perencana lokal bawaan karena algoritma ini memiliki beban komputasi yang rendah dan performa waktu nyata yang bagus. DWA menghasilkan lintasan yang aman dan layak secara waktu nyata dengan cara melakukan pengambilan sampel pada ruang kecepatan. Setiap sampel kecepatan hasil tersebut kemudian dievaluasi secara matematis berdasarkan fungsi biaya yang mencakup perhitungan keselarasan arah atau, jarak kedekatan dengan rintangan, dan efisiensi kecepatan.

Dalam proses pelacakan jalur, DWA bekerja dengan memprediksi posisi robot di masa depan berdasarkan masukan kecepatan saat ini. Algoritma ini kemudian mengkalkulasi deviasi arah dan jarak minimum terhadap rintangan terdekat sebagai acuan untuk memandu sistem dalam memilih lintasan terbaik. Setelah lintasan lokal terbaik terpilih, lapisan eksekusi kontrol akan menerjemahkan lintasan tersebut menjadi perintah kendali gerak yang konkret. Perintah

berupa kecepatan linier dan sudut ini dipublikasikan secara kontinu melalui antarmuka standar perintah ROS yaitu `cmd_vel`, yang pada akhirnya membentuk sistem kendali tertutup dengan base controller robot penggerak (Wei et al., 2025).

BAB III

METODOLOGI

3.1 Bahan dan Peralatan

Tugas akhir ini memanfaatkan model simulasi perangkat keras dan perangkat lunak yang dikonfigurasi dalam simulator Mujoco. Perangkat keras yang disebutkan merupakan model digital yang digunakan dalam simulasi, bukan perangkat fisik yang dioperasikan secara langsung.

3.1.1 Model robot hexapod

Model robot hexapod digunakan sebagai platform robot pada lingkungan simulasi MuJoCo. Model yang ditunjukkan pada gambar 3.1 merepresentasikan robot hexapod berkaki enam yang masing-masing dapat dikendalikan secara individual, sehingga mampu menyesuaikan postur dan posisi saat berpindah antar region atau lapisan *costmap*. Model robot ini dipilih karena sesuai dengan karakteristik gerak yang dibutuhkan untuk navigasi di medan tidak rata.



Gambar 3.1: Model platform navigasi yang digunakan dalam simulasi

3.1.2 Simulasi Sensor

Sensor LiDAR 2D pada lingkungan simulasi dikonfigurasi untuk mendeteksi rintangan di sekitar robot secara *real-time*. Sensor bekerja dengan memancarkan sinar laser virtual dan mengukur jarak ke objek di sekitarnya. Data hasil pembacaan digunakan sebagai masukan untuk membentuk *costmap* dan deteksi rintangan pada sistem navigasi. LiDAR yang digunakan adalah YDLiDAR T-mini-plus seperti yang ditunjuk pada gambar 3.2a. LiDAR yang digunakan dalam simulasi disetarakan dengan spesifikasi YDLiDAR T-mini-plus, yaitu frekuensi *scan* 12 Hz, *scan angle* 360°, resolusi sudut 0.54°, dan jarak maksimum 12 m.

BNO080 (gambar 3.2b) adalah modul IMU (*Inertial Measurement Unit*) 9-sumbu yang

mengintegrasikan magnetometer 3-sumbu, gyroscope 3-sumbu, dan akselerometer 3-sumbu dengan prosesor. Prosesor tersebut menjalankan algoritma *sensor fusion* secara internal untuk menggabungkan ketiga sensor tersebut, sehingga modul ini dapat menghasilkan estimasi orientasi yang stabil dan telah terkompensasi *drift*. BNO080 menyederhanakan pelacakan orientasi dengan menghasilkan data orientasi secara langsung, dalam bentuk sudut Euler untuk *heading*, *pitch*, dan *roll*.



(a) Sensor YDLiDAR T-mini-plus

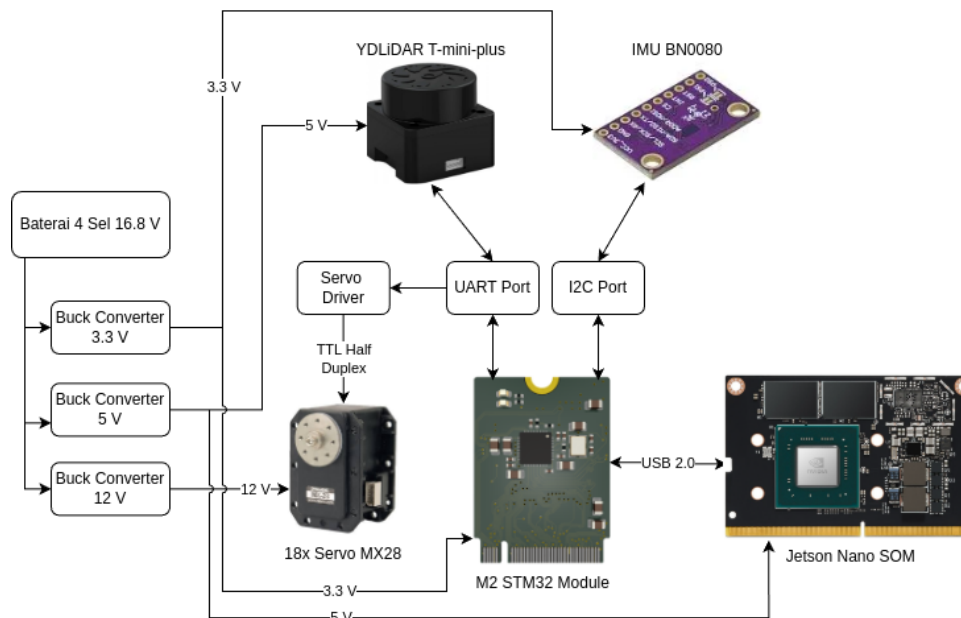


(b) Sensor IMU BNO080

Gambar 3.2: Referensi sensor yang digunakan dalam simulasi (Shenzhen, 2024)

3.1.3 Perangkat Keras

Gambar 3.3 menunjukkan diagram perangkat keras pada robot asli yang menjadi acuan pengembangan sistem navigasi dalam penelitian ini. Berbeda dengan model digital yang dijalankan di dalam simulator MuJoCo, diagram ini merepresentasikan komponen fisik sesungguhnya yang menyusun robot, yaitu Jetson Nano, STM32, sensor LiDAR 2D, dan IMU BNO080, beserta hubungan dan alur daya antar komponen tersebut. Konfigurasi perangkat keras pada robot asli ini digunakan sebagai dasar dalam menentukan spesifikasi sensor dan parameter sistem yang kemudian disetarakan ke dalam lingkungan simulasi.



Gambar 3.3: Diagram perangkat keras pada robot asli

3.1.4 Perangkat lunak pendukung

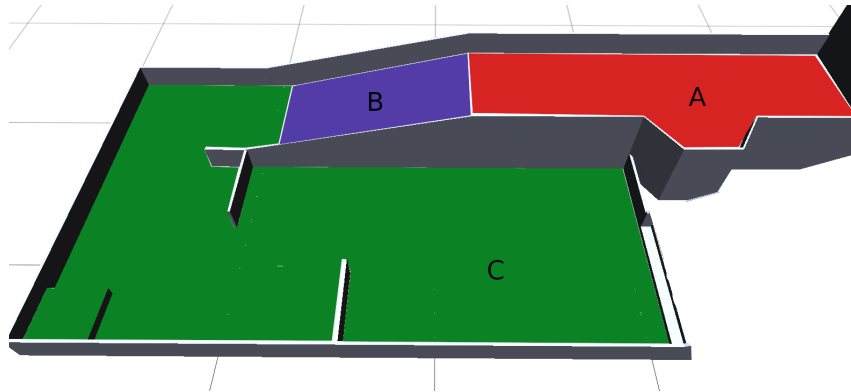
Penelitian ini menggunakan beberapa perangkat lunak pendukung yang saling terintegrasi untuk membangun sistem navigasi robot otonom, yaitu:

1. **Robot Operating System (ROS) Noetic**, merupakan *middleware* yang menyediakan kerangka kerja (*framework*) komunikasi antarproses dalam sistem robot. Pada penelitian ini, ROS Noetic digunakan sebagai basis integrasi seluruh komponen sistem, mulai dari pembacaan sensor LiDAR, lokalisasi ICP, pengelolaan *costmap*, perencanaan jalur, hingga kendali robot.
2. **Simulator MuJoCo (Multi-Joint dynamics with Contact)**, adalah *physics simulator* yang digunakan untuk menjalankan simulasi lingkungan dan dinamika robot. Seluruh pengujian dalam penelitian ini dilakukan di dalam MuJoCo, termasuk pergerakan robot hexapod, pembacaan sensor LiDAR, dan interaksi robot dengan lingkungan simulasi.
3. **RViz**, merupakan perangkat lunak visualisasi dari ROS yang digunakan untuk menampilkan data sensor, posisi robot, *costmap*, serta jalur navigasi secara grafis. Dengan RViz, pengguna dapat memonitor pergerakan robot dan memverifikasi apakah sistem navigasi dan lokalisasi berjalan dengan benar dalam lingkungan simulasi.
4. **ROS Navigation Stack**, adalah *library* yang menyediakan kerangka kerja untuk navigasi robot bergerak. Komponen utama yang digunakan dalam penelitian ini meliputi move base sebagai *action server* yang mengelola pipeline navigasi, serta *costmap2d* sebagai implementasi *layered costmap* yang menjadi dasar representasi lingkungan.

3.2 Pengolahan Lingkungan 3D Menjadi Map 2D

Sistem navigasi ini dirancang untuk beroperasi pada area yang petanya sudah diketahui sebelumnya (*known map*). Informasi lingkungan tersebut disimpan dalam bentuk *occupancy grid*. Tantangan utama yang dihadapi adalah bahwa data lingkungan yang dikumpulkan bersifat tiga dimensi (3D), sementara *occupancy grid* yang digunakan hanya memiliki dua dimensi (2D). Untuk menyelaraskan perbedaan ini, lingkungan 3D dibagi menjadi beberapa *region* yang lebih kecil, dan setiap *region* direpresentasikan secara terpisah dalam bentuk peta 2D. Sehingga, setiap *region* menjadi proyeksi 2D dari sub-bagian lingkungan 3D. Pembagian *region* dalam ruang 3D dapat dilihat pada Gambar 3.4.

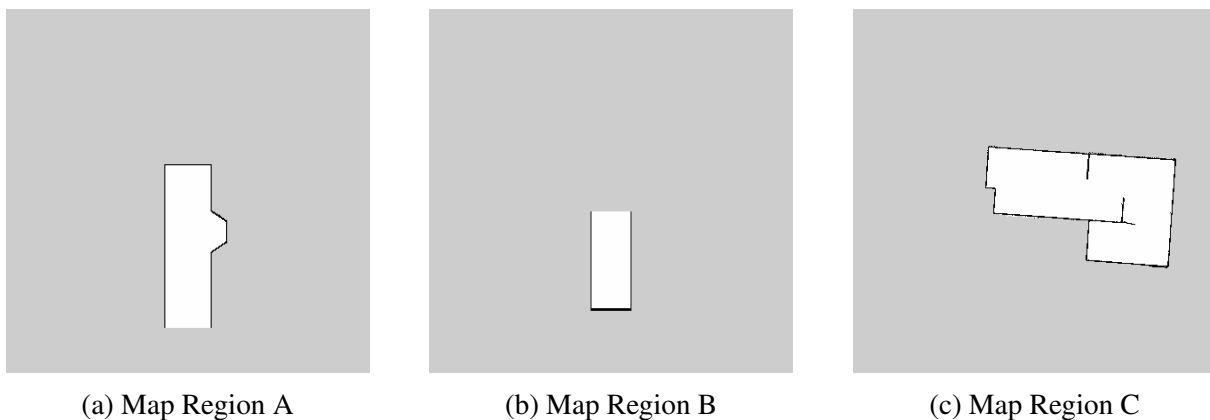
Selain peta utama, sistem navigasi juga membutuhkan peta transisi yang berfungsi sebagai jembatan penghubung antar *region*. Peta transisi adalah peta terpisah yang menyimpan informasi relasi spasial setiap *region* terhadap *global frame of reference* yang seragam, sehingga posisi absolut masing-masing *region* dalam koordinat global dapat diketahui. Dengan adanya peta transisi ini, robot dapat melakukan perpindahan antar *region* secara mulus selama navigasi, meskipun kedua *region* terpisah oleh area *unknown*.



Gambar 3.4: Lingkungan arena dalam 3D

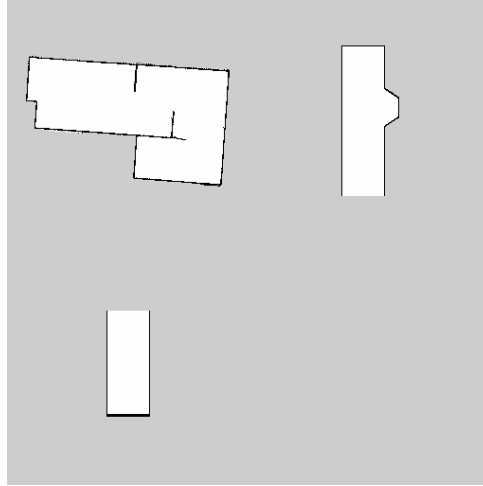
3.2.1 Map Region

Peta untuk setiap *region* diperoleh melalui proses *mapping* menggunakan Hector SLAM. Pada sistem ini, proses mapping dilakukan untuk setiap *region* secara terpisah. Robot dijalankan secara manual (*teleoperated*) di setiap area sambil mengumpulkan data *laser scan*. Hector SLAM memproses data tersebut untuk menghasilkan *occupancy grid* 2D masing-masing *region*. Peta yang dihasilkan kemudian diperiksa dan dilakukan penyesuaian manual jika terdapat artefak atau ketidaksesuaian dengan dimensi lingkungan asli. Hasil akhir peta untuk setiap *region* ditunjukkan pada Gambar 3.5.



Gambar 3.5: Map Occupancy Grid masing-masing region

Sistem navigasi tidak menggunakan peta individu dari masing-masing *region* secara terpisah, melainkan menggunakan satu peta utama yang merupakan hasil kombinasi seluruh *region* seperti di gambar 3.6. Dalam peta utama tersebut, setiap *region* dipisahkan oleh area *unknown* (wilayah yang tidak diketahui).



Gambar 3.6: Map Gabungan

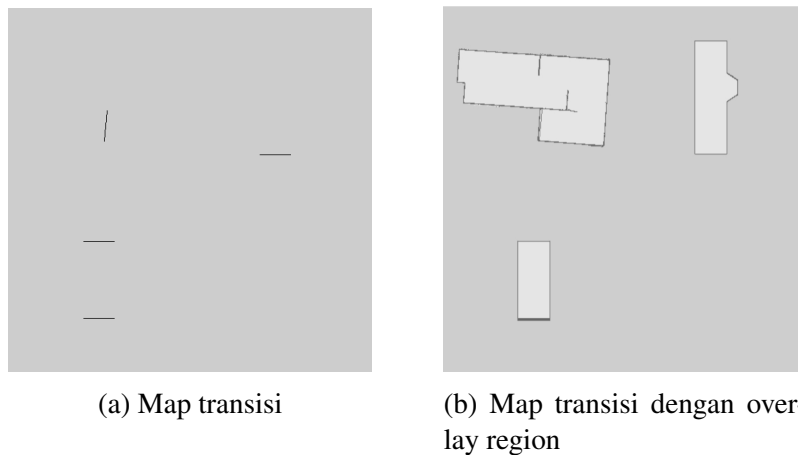
Untuk menghasilkan map gabungan, diperlukan deteksi Region of Interest (ROI) pada setiap peta pisah. ROI didefinisikan sebagai area yang mengandung piksel gelap (nilai $\leq$ threshold), yang mewakili obstacle atau peta yang bermakna. Untuk setiap peta, ROI diidentifikasi melalui:

$$\text{ROI} = \{(x, y) : I(x, y) < T\}$$

dimana, $I(x, y)$ adalah nilai intensitas piksel pada koordinat (x, y) , T adalah threshold.

Peta gabungan akhir tersusun dari beberapa bagian, dan setiap bagian ditempati oleh satu *region*. Banyaknya bagian tersebut ditentukan oleh jumlah peta *region* yang dimasukkan ke dalam proses penggabungan.

3.2.2 Map Transisi



Gambar 3.7: Map transisi antar region

Peta transisi digunakan untuk menandakan perbatasan antar *region* dalam peta gabungan. Peta ini dibuat melalui proses anotasi terhadap peta gabungan, dimana pengguna secara manual

menandai piksel-piksel yang mewakili perbatasan antar *region*. Hasil peta tersebut seperti gambar 3.7a dan dari overlay gambar 3.7b menunjukkan posisi garis transisi dibanding peta gabungan. Proses anotasi menghasilkan 'metadata' yang menyimpan informasi sebagai berikut: (1) pasangan *region* yang terhubung, (2) orientasi setiap *region* terhadap *global frame of reference* dalam bentuk *quaternion*, serta (3) posisi inisial robot pada setiap *region* dalam koordinat piksel.

Tabel 3.1: Daftar Koneksi pasangan

Pasangan	Region	Seed Entry (x, y)	Seed Exit (x, y)
0	A – B	(354, 206)	(106, 328)
1	B – C	(106, 436)	(135, 187)

Koneksi antar *region* bersifat linear, artinya setiap pasangan hanya menghubungkan dua *region* yang berdekatan secara langsung. Sebagai contoh, *region* A hanya dapat terhubung ke *region* B, dan *region* B hanya dapat terhubung ke *region* C. Tidak ada koneksi langsung dari *region* A ke *region* C. Setiap pasangan direpresentasikan oleh dua piksel, yaitu *Seed Entry* dan *Seed Exit*, yang menandai lokasi garis transisi pada peta transisi.

Tabel 3.2: Orientasi Region terhadap Global Frame

Region	Quaternion			
	x	y	z	w
A	0.0	0.0	0.0	1.0
B	0.0	0.0	0.0	1.0
C	0.0	0.0	-0.7046	0.7096

Orientasi digunakan untuk menyelaraskan arah peta setiap *region* dengan arah yang seragam dalam *global frame of reference*. Dalam kasus ini, *region* A dan B memiliki orientasi yang sama, sedangkan *region* C memiliki rotasi berbeda dengan A dan B.

Tabel 3.3: Posisi Inisial Robot

Parameter	Nilai pixel
Posisi inisial	(375, 75)

Posisi awal robot ditentukan secara manual dalam koordinat piksel, lalu dikonversi ke koordinat dunia menggunakan informasi orientasi dan resolusi peta. Posisi ini digunakan sebagai titik awal untuk dalam sistem navigasi.

3.2.3 Representasi Topologi Graph

Untuk memodelkan hubungan antar *region* secara formal, sistem navigasi merepresentasikan map transisi sebagai *topological graph* tak-berarah $G = (V, E)$. Setiap titik penting dalam sistem navigasi dinyatakan sebagai *node*, dan konektivitas antar titik tersebut dinyatakan

sebagai *edge*. Sistem ini digunakan untuk merencanakan lintasan dari posisi awal robot ke tujuan.

Tabel 3.4: Daftar node pada topological graph

Node	Label	Deskripsi	Region
v_0	Init	Posisi awal robot	A
v_1	Entry _{AB}	Titik transisi sisi A (pasangan 0)	A
v_2	Exit _{AB}	Titik transisi sisi B (pasangan 0)	B
v_3	Entry _{BC}	Titik transisi sisi B (pasangan 1)	B
v_4	Exit _{BC}	Titik transisi sisi C (pasangan 1)	C
v_5	Goal	Posisi tujuan	C

Setiap *node* $v_i \in V$ adalah titik acuan dalam koordinat dunia yang diperoleh dari metadata transisi (pixel transisi dan orientasi *region*). Himpunan *edge* E didefinisikan sebagai berikut:

$$E = \{(v_0, v_1), (v_1, v_2), (v_2, v_3), (v_3, v_4), (v_4, v_5)\} \quad (3.1)$$

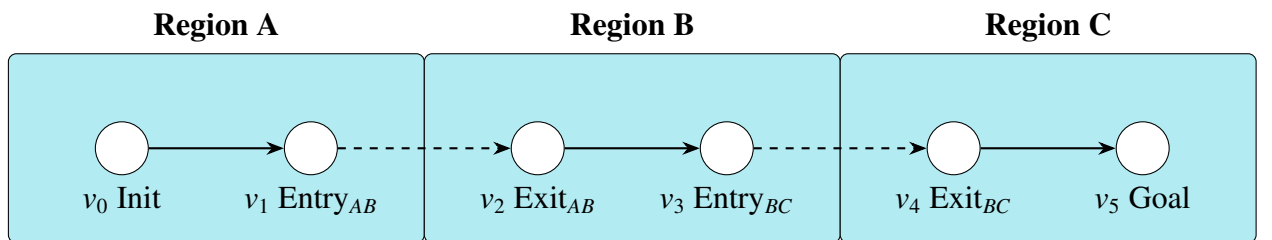
Edge dibagi menjadi dua tipe berdasarkan wilayah lintasannya:

1. **Intra-region:** $(v_0, v_1), (v_2, v_3), (v_4, v_5)$. *Edge* ini berada di dalam satu *region* dan dapat dilalui oleh navigasi standar.
2. **Inter-region** (transisi): (v_1, v_2) (pair 0, A–B) dan (v_3, v_4) (pair 1, B–C). *Edge* ini melintasi batas antar *region* dan memerlukan *region switcher* dan *path router* untuk melakukan navigasi.

Partisi *region* terhadap himpunan node adalah:

$$R_A = \{v_0, v_1\}, \quad R_B = \{v_2, v_3\}, \quad R_C = \{v_4, v_5\} \quad (3.2)$$

Representasi graph dari map transisi divisualisasikan pada Gambar 3.8.



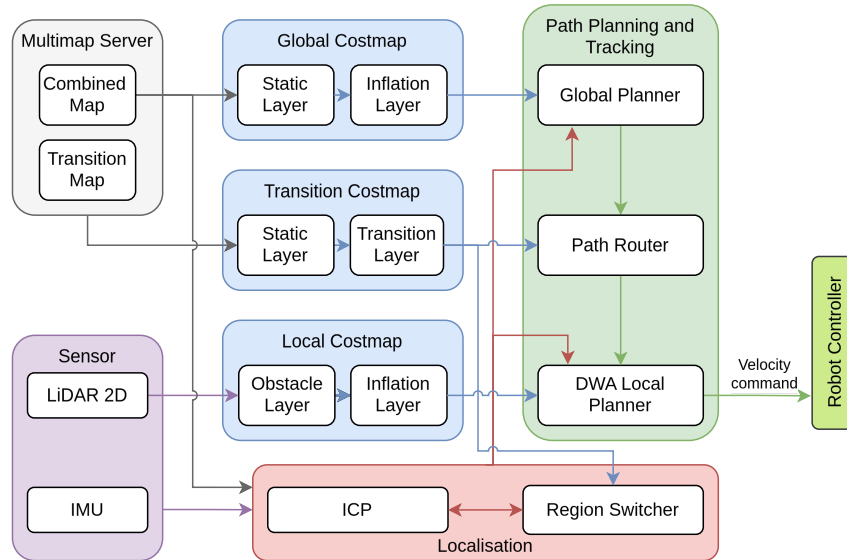
Gambar 3.8: Representasi graph dari map transisi

Dengan representasi graph ini, problem navigasi multi-*region* dapat direduksi menjadi pencarian lintasan pada G . Misalnya, untuk perjalanan dari posisi awal (v_0) ke tujuan (v_5) melewati *region* B, lintasan yang ditempuh adalah:

$$P = (v_0, v_1, v_2, v_3, v_4, v_5) \quad (3.3)$$

Lintasan ini berkorespondensi langsung dengan urutan *step* yang dihasilkan oleh *path router*, yaitu melewati pasangan 0 (A ke B) dan pasangan 1 (B ke C) secara berurutan. Karena graph G bersifat tak-berarah, lintasan yang sama berlaku untuk arah sebaliknya (C ke A).

3.3 Perancangan Sistem Navigasi



Gambar 3.9: Arsitektur sistem navigasi

Sistem navigasi ini berfungsi untuk merencanakan dan mengeksekusi pergerakan robot dari satu region ke region lain pada lingkungan yang telah diketahui. Keseluruhan relasi dan hubungan antar komponen dalam sistem ditunjukkan oleh gambar 3.9. Sistem dapat dibagi menjadi empat komponen utama, yaitu multimap server, costmap, lokalisasi, serta path planning dan tracking. Occupancy grid menyediakan representasi diskrit dari lingkungan. Costmap menyimpan informasi tersebut dalam bentuk peta biaya (costmap). Lokalisasi bertugas melacak posisi dan region robot secara real-time. Path planning dan tracking bertanggung jawab menghasilkan jalur antar region dan menjalankannya.

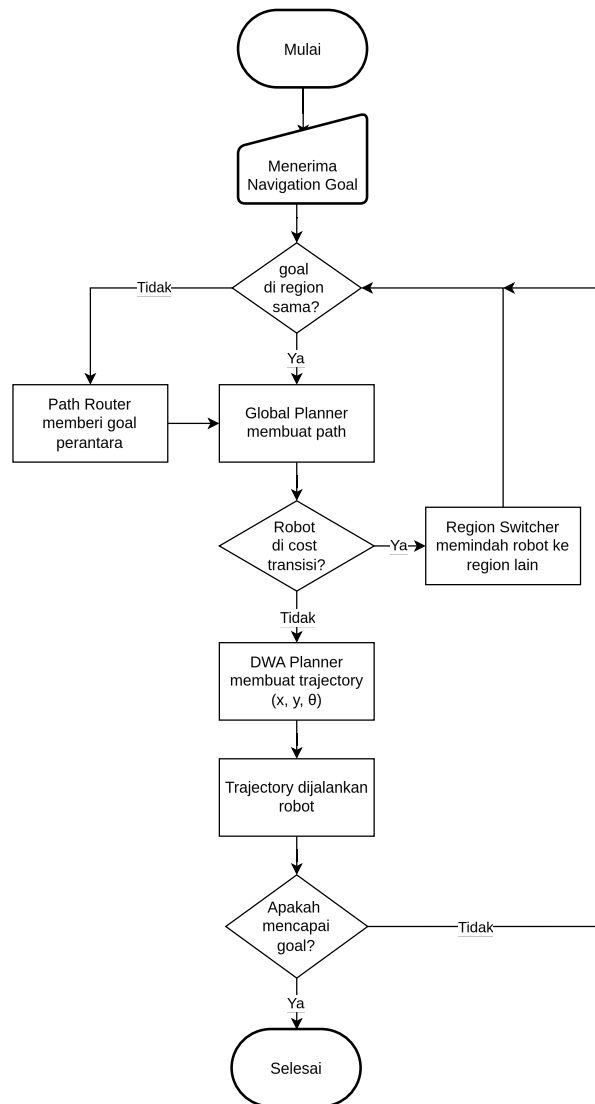
Data combined map (peta gabungan) digunakan untuk membangun global costmap, dimulai dari pembentukan static layer yang merepresentasikan obstacle yang diketahui dari peta. Selain itu, combined map juga menjadi basis bagi lokalisasi ICP untuk dikomparasi dengan hasil scan LiDAR guna memperkirakan pose robot. Sementara itu, map transisi digunakan sebagai sumber data bagi transition costmap, yang menyimpan informasi koneksi antar region.

Costmap dalam sistem ini dibagi menjadi tiga lapisan, yaitu global costmap, local costmap, dan transition costmap. Global dan local costmap sama-sama menggunakan inflation layer, namun dengan sumber data yang berbeda. Inflation layer pada global costmap dibangun berdasarkan static layer yang berasal dari obstacle tetap pada peta. Inflasi pada local costmap didasarkan pada obstacle layer yang diperbarui secara dinamis dari data scan LiDAR.

Global costmap digunakan oleh global planner untuk merencanakan path utama menuju tujuan. Untuk navigasi antar region, path router bertugas merutekan ulang jalur menuju zona transisi ketika tujuan berada di region yang berbeda. Hasil rutekan tersebut kemudian diteruskan ke dynamic window approach (DWA) local planner, yang menghasilkan trajectory berupa ke-

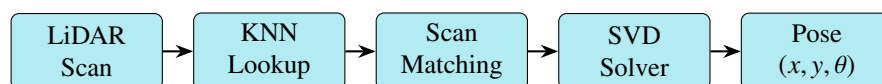
cepatan x , y , dan rotasi yang diberikan ke kontroler robot untuk dieksekusi.

Untuk memfasilitasi tracking, lokalisasi ICP dan region switcher bekerja sama melacak posisi koordinat robot serta region tempat robot berada saat ini. Ketika robot mencapai zona transisi, sistem masuk menjadi **mode transisi** untuk melakukan navigasi di perbatasan antar region dimana sensor LiDAR tidak bisa diandalkan. Karena region B adalah penghubung region A-C dan merupakan bidang miring sehingga digunakan sensor IMU untuk mendeteksi kemiringan. Setelah batas pitch tertentu region switcher melakukan perpindahan region dan memperbarui posisi awal ICP pada region yang baru. Gambar 3.10 menunjukkan diagram alur navigasi.



Gambar 3.10: Diagram alur navigasi antar lantai

3.3.1 Lokalisasi Iterative Closest Point (ICP)



Gambar 3.11: Diagram alir proses ICP

Sistem navigasi ini menggunakan algoritma Iterative Closest Point (ICP) yang digunakan untuk memperkirakan koreksi pose robot terhadap static occupancy grid map. ICP bekerja dengan mencocokkan titik-titik dari laser scan dengan titik-titik yang sesuai pada peta, lalu menghitung transformasi optimal yang meminimalkan jarak antara pasangan titik tersebut. Proses ini iteratif, di mana setiap iterasi memperbaiki estimasi pose robot hingga konvergen ke solusi yang stabil. Sebelum ICP, dilakukan prekomputasi KNN untuk mempercepat pencarian pasangan titik selama proses ICP. Alur garis besar ditunjukkan pada gambar 3.11.

3.3.1.1 Prekomputasi KNN

Sistem ini melakukan prekomputasi atas semua kemungkinan pasangan titik (sensor point $\rightarrow$ map point) saat menerima map baru, lalu menyimpan informasi tersebut di dalam hash table. Pendekatan ini dimungkinkan karena occupancy grid bersifat diskrit dan diketahui secara penuh. Setiap sel bebas di dekat tembok dapat diketahui sebelumnya K buah sel rintangan terdekatnya. Proses ini disebut interpretasi map dan terjadi sekali saat startup, bukan saat runtime. Hasilnya lookup KNN saat runtime menjadi $O(1)$.

Untuk setiap sel rintangan (occupied) pada indeks i_{occ} :

1. Cek kejangkauan menggunakan Moore neighbourhood 3×3 (8 tetangga terdekat). Jika tidak ada satupun sel bebas di antara tetangganya, maka sel rintangan tersebut berada di dalam tembok (interior) dan tidak akan pernah terdeteksi oleh laser hanya mengenai permukaan tembok, bukan interiornya. Sel seperti ini diabaikan untuk menghemat memori dan komputasi.
2. Iterasi area sekitar sel rintangan yang dapat dicapai. Semakin besar r , semakin banyak sel yang akan memiliki KNN precomputed, tetapi juga semakin besar ukuran hash table.
3. Untuk setiap sel tetangga pada indeks i_{nb} yang berada dalam radius r , *world coordinates* dari sel rintangan i_{occ} dimasukkan ke dalam daftar KNN milik i_{nb} . Daftar ini dijaga tetap terurut berdasarkan jarak Euclidean dan dibatasi maksimal K entry. Jika sudah penuh dan jarak lebih besar dari entry terjauh, data diabaikan, namun jika jarak lebih kecil, entry terjauh dihapus dan data baru dimasukkan pada posisi yang tepat.

3.3.1.2 Proses Data LiDAR

Data scan dari sensor LiDAR diproses terlebih dahulu dan disimpan dalam format seperti berikut. Untuk scan dengan sinar N pada sudut $\theta_0, \theta_1, \dots, \theta_{N-1}$:

$$\mathbf{C} = \begin{bmatrix} \cos \theta_0 & \cos \theta_1 & \cdots & \cos \theta_{N-1} \\ \sin \theta_0 & \sin \theta_1 & \cdots & \sin \theta_{N-1} \end{bmatrix} \quad (3.4)$$

Cache hanya dihitung ulang ketika parameter scan (jumlah sinar, rentang sudut, increment) berubah.

Diberikan pembacaan vektor $r_0, r_1, \dots, r_{N-1}$:

$$\mathbf{S} = \begin{bmatrix} r_0 \cos \theta_0 & r_1 \cos \theta_1 & \cdots & r_{N-1} \cos \theta_{N-1} \\ r_0 \sin \theta_0 & r_1 \sin \theta_1 & \cdots & r_{N-1} \sin \theta_{N-1} \end{bmatrix} \quad (3.5)$$

S adalah hasil *scan point cloud* dalam koordinat cartesian, hasil konversi dari data laser mentah ke titik-titik 2D.

3.3.1.3 Loop Utama

Algorithm 1 ICP Single Iteration

Require: $scan$	▷ N points dalam sensor frame
Ensure: new_to_old	▷ transform_t
1: $T \leftarrow Identity$	
2: for $iteration = 1$ to max_iter do	
3: $random_scan \leftarrow sampler(scan)$	▷ subsample dengan random stride
4: $data \leftarrow T \cdot random_scan$	▷ transform ke map frame
5: $M \leftarrow matches(data, knn)$	▷ cari correspondences
6: $w \leftarrow get_weights(M)$	▷ weight setiap match
7: $T_update \leftarrow point_to_map(M, w)$	▷ SVD solver
8: $T \leftarrow T_update \cdot T$	▷ akumulasi
9: if $converged(T_update)$ then	
10: break	
11: end if	
12: end for	
13: return T	

3.3.1.4 Correspondence Matching

Correspondence matching adalah langkah yang memasangkan setiap titik dari laser scan dengan titik terdekat dari map. Proses ini dilakukan dengan memanfaatkan hash table KNN yang sudah diprekomputasi sebelumnya. Hasil akhirnya digunakan untuk menghitung transformasi optimal melalui SVD solver.

Untuk setiap sensor point s_j :

1. Cek apakah s_j berada di dalam map bounds. Jika tidak, lewati s_j .
2. melakukan lookup ke hash table untuk mendapatkan daftar hingga K titik map terdekat yang mungkin menjadi pasangan
3. Duplikasi s_j untuk setiap nearest neighbors-nya, menghasilkan $(s_j, m_{j,1}), (s_j, m_{j,2}), \dots, (s_j, m_{j,K})$. Ini memperluas N sensor points menjadi hingga $N \times K$ pasangan yang cocok.

3.3.1.5 Solusi SVD

Solusi SVD ini digunakan untuk menghitung transformasi yang memetakan titik-titik dari laser scan ke titik-titik yang sesuai di map, berdasarkan pasangan yang ditemukan pada langkah correspondence matching.

Diberikan M pasangan $\{s_i\}$ (sensor) dan $\{m_i\}$ (map) dengan weights $\{w_i\}$, kita mencari rigid transform $T \in SE(2)$ yang meminimalkan transformasi. $SE(2)$ berarti transformasi yang mempertahankan jarak dan orientasi.

$$T^* = \arg \min_{R, t} \sum_{i=1}^M w_i \|m_i - (Rs_i + t)\|^2 \quad (3.6)$$

Langkah 1: Kalkulasi *Weighted Centroids*

$$\bar{s} = \frac{\sum_{i=1}^M w_i s_i}{\sum_{i=1}^M w_i}, \quad \bar{m} = \frac{\sum_{i=1}^M w_i m_i}{\sum_{i=1}^M w_i} \quad (3.7)$$

Langkah 2: Centering (pindahkan titik ke pusat koordinat)

$$s'_i = s_i - \bar{s}, \quad m'_i = m_i - \bar{m} \quad (3.8)$$

Langkah 3: Cross-Covariance Matrix

$$H = \sum_{i=1}^M w_i m'_i (s'_i)^\top = M W S^\top \quad (3.9)$$

dimana $S, M \in \mathbb{R}^{2 \times M}$ adalah matriks yang kolomnya berisi titik-titik yang sudah di-center, dan $W = \text{diag}(w_1, \dots, w_M)$.

Langkah 4: Singular Value Decomposition

$$H = U \Sigma V^\top \quad (3.10)$$

dengan $U, V \in \mathbb{R}^{2 \times 2}$ orthonormal dan $\Sigma = \text{diag}(\sigma_1, \sigma_2)$.

Langkah 5: Rotasi Optimal

$$R = UV^\top \quad (3.11)$$

Jika $\det(R) < 0$ (menghasilkan refleksi, bukan rotasi murni), koreksi dengan membalik tanda baris pertama $V^\top$:

$$\text{if } \det(R) < 0 : \quad V_{0,:}^\top \leftarrow -V_{0,:}^\top, \quad R = UV^\top \quad (3.12)$$

Langkah 6: Translasi Optimal

$$t = \bar{m} - R\bar{s} \quad (3.13)$$

Langkah 7: Susun Rigid Transform 2D

$$T_{\text{update}} = \begin{bmatrix} R & t \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} \cos \Delta\theta & -\sin \Delta\theta & t_x \\ \sin \Delta\theta & \cos \Delta\theta & t_y \\ 0 & 0 & 1 \end{bmatrix} \quad (3.14)$$

Setelah iterasi, cek apakah transform update cukup kecil:

$$\text{converged} \iff \|t\| < t_{\text{norm}} \wedge \left| \arctan 2(R_{1,0}, R_{0,0}) \right| < r_{\text{norm}} \quad (3.15)$$

Untuk mempercepat, hanya $1/s$ dari scan points yang digunakan:

1. Pilih offset acak $o \in [0, s - 1]$
2. Ambil setiap point ke- s mulai dari o :

$$p'_k = p_{o+k \cdot s}, \quad k = 0, 1, \dots, \left\lfloor \frac{N}{s} \right\rfloor - 1 \quad (3.16)$$

3.3.2 Costmap

Costmap adalah representasi grid 2D yang menyimpan informasi biaya navigasi. Setiap sel dalam *costmap* memiliki nilai numerik yang mencerminkan tingkat kesulitan atau risiko untuk dilalui. Nilai biaya yang rendah menandakan area yang aman untuk dilalui, sedangkan nilai biaya yang tinggi menandakan area yang berbahaya atau terlarang. Nilai biaya ini dipengaruhi oleh berbagai faktor, seperti keberadaan rintangan statis, rintangan dinamis, dan zona transisi antar region. Nilai konstanta *costmap* yang digunakan dalam sistem ini ditunjukkan pada Tabel 3.5.

Tabel 3.5: Nilai konstanta costmap

Nama Konstanta	Nilai	Makna
NO_INFORMATION	255	Tidak diketahui (<i>unknown</i>)
LETHAL_OBSTACLE	254	Obstacle mematikan
INSCRIBED_INFLATED_OBSTACLE	253	Batas terluar inflasi obstacle
INFLATED_OBSTACLE	1-252	Inflasi obstacle
TRANSITION_CELL	1-5	Area transisi antar region
FREE_SPACE	0	Ruang bebas

Setiap sel dalam *costmap* disimpan dalam bentuk array 1D agar memudahkan akses dan pemrosesan. Konversi dari koordinat grid 2D (m_x, m_y) ke indeks array 1D dilakukan dengan persamaan berikut:

$$i = m_y \cdot W + m_x \quad (3.17)$$

dengan W adalah lebar grid dalam jumlah sel, m_x adalah kolom, dan m_y adalah baris.

Untuk menghubungkan koordinat dunia (dalam meter) dengan koordinat grid (dalam sel), digunakan parameter origin grid (o_x, o_y) dan resolusi r yang menyatakan panjang satu sisi sel dalam meter. Konversi dari koordinat dunia (w_x, w_y) ke koordinat grid:

$$m_x = \left\lfloor \frac{w_x - o_x}{r} \right\rfloor, m_y = \left\lfloor \frac{w_y - o_y}{r} \right\rfloor \quad (3.18)$$

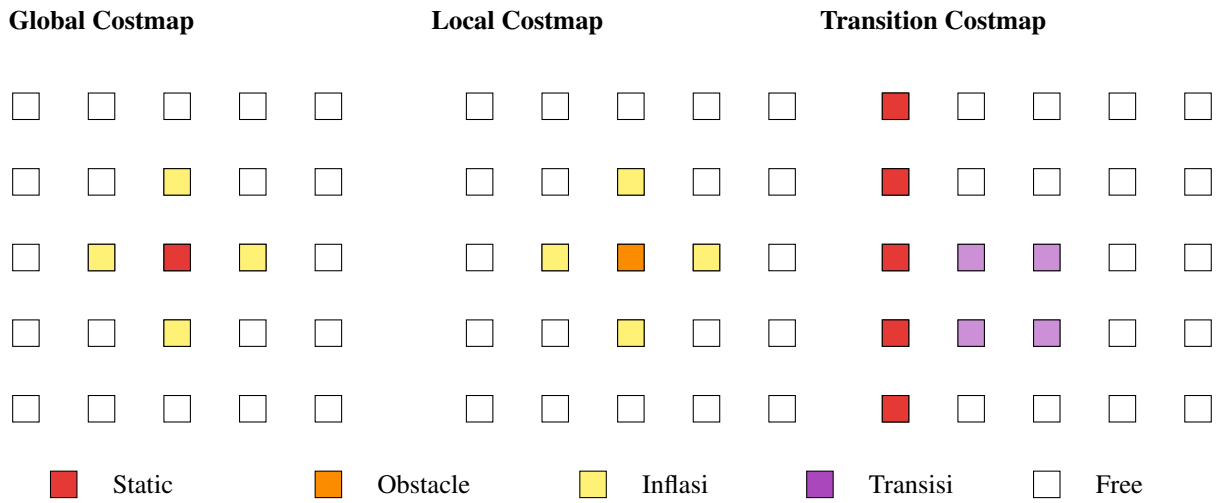
Konversi dari koordinat grid ke koordinat dunia:

$$w_x = o_x + \left(m_x + \frac{1}{2}\right) \cdot r, w_y = o_y + \left(m_y + \frac{1}{2}\right) \cdot r \quad (3.19)$$

Dalam sistem ini, terdapat tiga jenis *costmap* yang masing-masing memiliki komposisi *layer* berbeda sesuai dengan fungsinya. Komposisi tersebut dirangkum pada Tabel 3.6, dan bentuk visualisasi ditunjukkan oleh gambar 3.12.

Tabel 3.6: Komposisi layer pada masing-masing *costmap*

Lapisan Costmap	Tippe Layer	Fungsi
Global Costmap	Static Layer	Menyimpan obstacle tetap dari peta
	Inflation Layer	Memperluas obstacle dengan gradient cost
Local Costmap	Obstacle Layer	Mendeteksi rintangan dinamis dari scan LiDAR
	Inflation Layer	Memperluas obstacle dengan gradient cost
Transition Costmap	Static Layer	Menyimpan obstacle tetap dari peta
	Transition Layer	Menyimpan area transisi antar region



Gambar 3.12: Visualisasi grid masing-masing *costmap*

Global costmap adalah *costmap* yang mencakup seluruh area yang diketahui dari peta. *Costmap* ini digunakan oleh *global planner* untuk merencanakan jalur dari posisi robot menuju tujuan dalam skala luas. *Global costmap* terdiri dari *static layer* yang menyimpan informasi rintangan tetap dari peta, serta *inflation layer* yang memperluas area rintangan dengan *gradient cost* untuk menghindari tabrakan. *Costmap* ini tidak berubah secara dinamis selama navigasi berlangsung, kecuali jika terdapat pembaruan peta dari luar.

Local costmap adalah *costmap* yang mencakup area di sekitar robot dalam radius tertentu. *Costmap* ini digunakan oleh *local planner* untuk perencanaan gerak jangka pendek, seperti menghindari rintangan yang baru terdeteksi. *Local costmap* terdiri dari *obstacle layer* yang mendeteksi rintangan dinamis dari data *scan* LiDAR secara *real-time*, serta *inflation layer* untuk

menjaga jarak aman dari rintangan tersebut. Berbeda dengan *global costmap*, *local costmap* bergerak mengikuti posisi robot (*rolling window*) sehingga area yang dicakup selalu berpusat di sekitar robot.

Transition costmap adalah *costmap* yang menggabungkan data transisi dengan peta statis. *Transition costmap* memiliki *static layer* yang menyimpan informasi rintangan tetap dari peta, serta *transition layer* yang memberikan nilai biaya khusus pada sel-sel yang berada di zona transisi. Nilai biaya ini digunakan oleh *region switcher* untuk mendeteksi kapan robot memasuki area perbatasan antar *region* sehingga dapat melakukan lompatan pose ke *region* tujuan.

Local costmap menggunakan mekanisme *rolling window*, yaitu grid bergerak mengikuti posisi robot. Ketika robot berpindah, origin grid digeser sehingga robot selalu berada di dekat pusat *costmap*. Pergeseran origin dihitung dengan:

$$\Delta_x = \lfloor \text{new_origin}_x - \text{old_origin}_x \rfloor, \quad \Delta_y = \lfloor \text{new_origin}_y - \text{old_origin}_y \rfloor \quad (3.20)$$

Setelah origin digeser, array *costmap* digeser sebesar (Δ_x, Δ_y) dan sel-sel baru yang muncul di area yang sebelumnya belum tercakup diisi dengan informasi.

3.3.2.1 Static Layer

Static layer adalah lapisan *costmap* paling dasar yang menyimpan informasi *known obstacle* atau tembok yang sudah diketahui. Lapisan *costmap* ini mengkonversi setiap sel dari nilai occupancy grid (0-100) ke nilai biaya *costmap* internal (0-255). Sel yang tidak diketahui (unknown) diberi biaya tertinggi, sel bebas diberi biaya terendah, dan sel yang terisi (occupied) diberi biaya tinggi.

$$C_{\text{internal}} = \begin{cases} 255 & \text{if } OG = -1 \text{ (unknown)} \\ 0 & \text{if } OG = 0 \text{ (free)} \\ 254 & \text{if } OG \geq 100 \text{ (occupied)} \end{cases} \quad (3.21)$$

3.3.2.2 Obstacle Layer

Obstacle layer adalah lapisan yang menangani rintangan dinamis atau rintangan yang baru terdeteksi selama navigasi. Berbeda dengan *static layer* yang bersumber dari peta tetap, informasi pada lapisan ini diperoleh secara *real-time* dari data *scan* sensor LiDAR. Lapisan ini memiliki batas jangkauan maksimum yang disesuaikan dengan spesifikasi sensor, sehingga hanya rintangan dalam radius pandang yang direpresentasikan.

Proses pembaruan *obstacle layer* dilakukan dalam beberapa langkah berikut:

1. Konversi endpoint

Untuk setiap berkas sinar laser dengan jarak r dan sudut θ , hitung posisi ujung sinar (*endpoint*) dalam koordinat sensor:

$$(x_{\text{sensor}}, y_{\text{sensor}}) = (r \cos \theta, r \sin \theta) \quad (3.22)$$

2. Transformasi koordinat

Endpoint ditransformasikan dari *frame* sensor ke *frame* peta menggunakan TF.

3. Konversi ke grid

Koordinat dunia (x, y) hasil transformasi dikonversi ke indeks sel *costmap* menggunakan persamaan 3.18.

4. Markasi obstacle

Sel yang ditempati oleh endpoint diberi nilai lethal obstacle (254), menandakan bahwa area tersebut tidak dapat dilalui.

Selain memarkir *endpoint* sebagai obstacle, sistem juga perlu memastikan bahwa area antara robot dan *endpoint* tidak terdeteksi sebagai rintangan. Untuk itu digunakan algoritma *ray tracing* Bresenham 2D. Algoritma ini menelusuri garis lurus dari posisi robot ke setiap *endpoint* dan membersihkan (*clear*) semua sel yang dilewati:

$$C_{\text{local}}(m_x, m_y) = 0 \quad \forall \text{ sel di sepanjang ray, kecuali endpoint} \quad (3.23)$$

Algoritma Bresenham menentukan sel-sel yang mendekati garis lurus antara dua titik grid. Diberikan titik awal (x_0, y_0) dan titik akhir (x_1, y_1) , parameter inisialisasi:

$$\Delta_x = |x_1 - x_0|, \quad \Delta_y = |y_1 - y_0|, \quad \text{err} = \Delta_x - \Delta_y \quad (3.24)$$

Algorithm 2 Bresenham Raytracing

```
1: while  $x \neq x_1 \vee y \neq y_1$  do
2:    $e_2 \leftarrow 2 \cdot \text{err}$ 
3:   if  $e_2 > -\Delta_y$  then
4:      $\text{err} \leftarrow \text{err} - \Delta_y$ 
5:      $x \leftarrow x + s_x$ 
6:   end if
7:   if  $e_2 < \Delta_x$  then
8:      $\text{err} \leftarrow \text{err} + \Delta_x$ 
9:      $y \leftarrow y + s_y$ 
10:  end if
11:  raytrace_function( $x, y$ )
12: end while
```

Setelah proses *ray tracing* selesai, *costmap* lokal memiliki informasi terkini tentang area yang aman dan area yang terhalang rintangan.

3.3.2.3 Inflation Layer

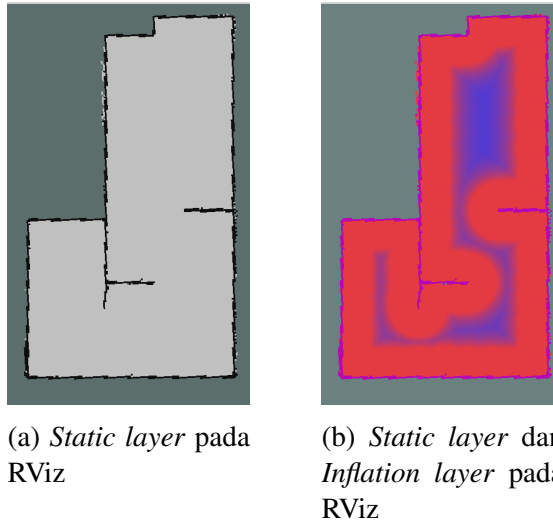
Inflation layer berperan penting dalam memastikan pergerakan robot dapat menghindari berhimpitan dengan rintangan. Lapisan ini menciptakan zona inflasi di sekitar setiap rintangan yang terdeteksi dan memiliki nilai harga navigasi yang tinggi. Radius zona inflasi ini variabel yang ditentukan berdasarkan dimensi fisik atau *footprint* robot sehingga mencegah robot terlalu dekat dengan rintangan dan berpotensi menabrak. Lapisan ini secara efektif ”memperbesar” rintangan di *costmap*, sehingga algoritma perencanaan jalur akan menjaga jarak aman dari objek-objek tersebut dari robot.

Lapisan inflasi berfungsi seperti safe distance minimum robot dari rintangan, namun dapat bekerja lebih dinamis karena memperhitungkan orientasi dan bentuk badan robot. Dengan bentuk robot yang unik, radius zona inflasi dapat berubah secara adaptif sesuai orientasi robot terhadap rintangan.

Layer ini memperluas obstacle dengan gradient cost, sehingga perencana path dapat menjaga jarak dari obstacle. Untuk setiap obstacle cell, terapkan cost ke semua cell dalam radius inflasi R . Cost pada jarak d dari obstacle dihitung dengan fungsi eksponensial:

$$C(d) = \begin{cases} 253, & \text{if } d \leq r_{\text{inscribed}} \\ 252 \cdot e^{-\alpha \cdot (d - r_{\text{inscribed}})}, & \text{if } r_{\text{inscribed}} < d \leq R \\ 0, & \text{if } d > R \end{cases} \quad (3.25)$$

dimana: d = jarak Euclidean dari cell ke obstacle terdekat, R = inflation radius (jarak maksimum inflasi), $r_{\text{inscribed}}$ = inscribed radius (radius footprint robot), dan α = scaling factor (mengontrol kecuraman kurva cost). Perbedaan sebelum dan sesudah static layer ditambah inflation layer dapat ditunjukkan gambar 3.13.



Gambar 3.13: Visualisasi *global costmap* pada RViz

3.3.2.4 Transition Layer

Transition layer adalah lapisan yang menambahkan data area transisi antar *region* ke dalam *costmap*. Lapisan ini memiliki dua jenis nilai *cost* dengan fungsi yang saling melengkapi. Nilai $T = 1$ menandai garis transisi utama tempat robot akan melakukan lompatan pose. Nilai $E = 5$ membentuk zona buffer di sekitar garis transisi, sehingga robot dapat mendeteksi bahwa ia sedang mendekati batas *region* sebelum benar-benar mencapai garis transisi.

Nilai $T = 1$ berasal dari peta transisi yang dimuat melalui *topic* terpisah. Data ini berupa file PGM dengan nilai piksel 0–255 yang dikonversi ke *cost* internal:

$$C_{\text{internal}} = \begin{cases} 255 & \text{if } OG = -1 \text{ (unknown)} \\ 0 & \text{if } OG = 0 \text{ (free)} \\ 1 & \text{if } OG \geq 100 \text{ (occupied)} \end{cases} \quad (3.26)$$

Nilai $E = 5$ dihasilkan melalui ekspansi dari sel-sel transisi menggunakan algoritma *Breadth-First Search* (BFS). Seluruh sel dengan nilai $T = 1$ digunakan sebagai *seed* awal. BFS menelusuri tetangga dalam arah 4-way hingga radius R sel. Setiap sel yang dikunjungi dan bukan merupakan *seed* diberi nilai $E = 5$. Ekspansi berhenti jika bertemu sel dengan *cost* lethal obstacle (254) sehingga area transisi tidak meluas ke dalam rintangan. Pseudocode algoritma ini ditunjukkan pada Algorithm 3.

Algorithm 3 BFS Expansion untuk Transition Layer

Require: *map* (grid transisi), R (radius ekspansi dalam sel)

Ensure: *cost* terisi dengan nilai $E = 5$ pada sel buffer

```

1:  $q \leftarrow$  antrian kosong
2: for setiap sel  $(x, y)$  dengan  $map(x, y) = T$  do
3:    $q.push((x, y))$ 
4:    $distance[x][y] \leftarrow 0$ 
5: end for
6: while  $q$  tidak kosong do
7:    $(x, y) \leftarrow q.front()$ 
8:    $q.pop()$ 
9:    $d \leftarrow distance[x][y]$ 
10:  if  $d \geq R$  then
11:    continue
12:  end if
13:  for setiap tetangga  $(nx, ny)$  dalam arah 4-way do
14:    if  $(nx, ny)$  di luar batas grid then
15:      continue
16:    end if
17:    if  $cost[nx][ny] = 254$  then
18:      continue
19:    end if
20:    if  $distance[nx][ny]$  sudah terdefinisi then
21:      continue
22:    end if
23:     $distance[nx][ny] \leftarrow d + 1$ 
24:    if  $map(nx, ny) \neq T$  then
25:       $cost[nx][ny] \leftarrow E$ 
26:    end if
27:     $q.push((nx, ny))$ 
28:  end for
29: end while

```

Ilustrasi ekspansi transisi dengan radius $R = 2$ sel:

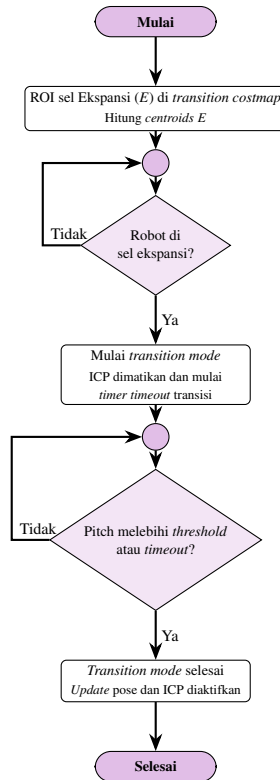
Sebelum ekspansi	Setelah ekspansi
.	. E E E .
. . T . .	E E T E E
. T T T .	E T T T E
. . T . .	E E T E E
.	. E E E .

(3.27)

dengan $T = 1$ (garis transisi) dan $E = 5$ (buffer ekspansi). Kedua nilai ini dideteksi oleh *region switcher*. Saat robot memasuki zona ekspansi ($E = 5$), *transition mode* diaktifkan; lompatan dieksekusi setelah *timer* atau *pitch trigger* terpenuhi.

Tiap *region* dalam peta memiliki metadata transisi yang menyimpan informasi relasi antar peta. Informasi tersebut mencakup orientasi relatif terhadap orientasi global yang dijadikan acuan.

3.3.3 Region Switcher



Gambar 3.14: Diagram alir region switcher

Region switcher bertugas mendeteksi kapan robot memasuki zona transisi, menghitung parameter lompatan, dan melakukan perpindahan *region* beserta pembaruan pose robot, dengan alur kerja seperti di gambar 3.14. Region switcher menggunakan *transition costmap* yang menyediakan data zona transisi terpisah dari *global costmap*, serta berkoordinasi dengan *path router* untuk navigasi multi-region yang mulus. Mekanisme ini berkaitan langsung dengan *topological graph* yang telah dijelaskan pada Subbab 3.2.3. Pada graph $G = (V, E)$, *region switcher* menangani *edge* inter-region (v_1, v_2) dan (v_3, v_4) . Setiap *edge* ini menghubungkan dua

region berbeda dan tidak dapat dilalui oleh *global planner* biasa sehingga lompatan pose harus dilakukan secara eksplisit.

3.3.3.1 Deteksi Data Transisi

Transition costmap mengandung sel ekspansi ($E = 5$) yang menandai area buffer di sekitar batas antar *region*. *Region switcher* melakukan pemindaian *rectangular window* sebesar ± 80 *cell* di sekitar *seed* untuk menemukan seluruh sel dengan nilai *cost* $E = 5$:

$$\text{untuk } dx, dy \in [-W, W] : \quad \text{jika } c(n_x, n_y) = E \rightarrow \text{centroid } E \quad (3.28)$$

Hasil pemindaian menghasilkan centroid E :

$$\mathbf{c}_E = \frac{1}{N_E} \sum_{i=1}^{N_E} (x_i, y_i) \quad (3.29)$$

Centroid E digunakan secara internal oleh *region switcher* sebagai tujuan lompatan.

3.3.3.2 Pelacakan Region

Peta gabungan (*combined map*) dipisah menjadi beberapa *region* (A, B, C). Untuk menentukan *region* tempat robot berada, dilakukan pencarian *node* terdekat dari *topological graph*. Setiap *node* terletak di centroid area transisi pada *region* masing-masing. Robot berada di *region* yang memiliki *node* paling dekat dengan posisinya saat ini.

3.3.3.3 Transition Mode

Ketika robot memasuki zona ekspansi ($E = 5$), *region switcher* mengaktifkan *transition mode*. Mekanisme ini membantu robot melewati batas antar *region* di mana lokalisasi ICP tidak dapat diandalkan. Selama *transition mode*, ICP dinonaktifkan sementara agar pose robot tidak berubah akibat koreksi selama proses transisi.

Eksekusi lompatan dalam *transition mode* dipicu oleh dua mekanisme:

1. *Pitch-based detection*

Jika robot menggunakan IMU dan *region* tujuan adalah *slope* (*region* A/C datar, *region* B *slope*), perubahan *pitch* langsung memicu lompatan:

$$|\theta_{\text{pitch}} - \theta_{\text{baseline}}| \geq \theta_{\text{threshold}} \quad (3.30)$$

θ_{baseline} diambil saat *transition mode* dimulai, dengan $\theta_{\text{threshold}} = 12^\circ$. Batas ini memungkinkan lompatan lebih awal saat robot mulai menaiki atau menuruni *slope*, tanpa perlu menunggu *timer* selesai.

2. *Timer-based*

Jika *pitch* tidak terdeteksi, lompatan dipicu setelah *timer* habis. Durasi dihitung berdasarkan radius ekspansi dan kecepatan robot, dengan faktor $2 \times$ karena robot melewati zona ekspansi di sisi masuk dan sisi keluar:

$$t_{\text{wait}} = \frac{2 \cdot r_{\text{expansion}}}{|\vec{v}|} \quad (3.31)$$

dengan $r_{\text{expansion}} = 0,45$ m pada konfigurasi dan $|\vec{v}| = \max(\sqrt{v_x^2 + v_y^2}, 0,01)$.

Timer-based transition tetap berfungsi sebagai *safety fallback* jika *pitch* tidak terdeteksi.

3.3.3.4 Eksekusi Lompatan

Saat *timer* habis atau *pitch trigger* terpenuhi, region switcher mengeksekusi lompatan dalam beberapa langkah berikut:

1. Perhitungan posisi lompatan

Robot melompat langsung ke centroid E pada *node* berikutnya dalam urutan *topological graph*. Robot melompat langsung ke pusat zona ekspansi *node* tujuan.

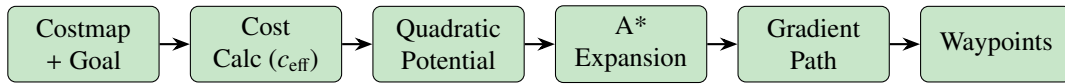
2. Orientasi lompatan

Jika *region* tujuan memiliki orientasi yang didefinisikan dalam *metadata* transisi, orientasi robot disesuaikan menggunakan *quaternion*:

$$\mathbf{q}_{\text{new}} = \mathbf{q}_{\text{region}} \cdot \mathbf{q}_{\text{current}} \quad (3.32)$$

Tanda $\cdot$ adalah perkalian *quaternion* (Hamilton product). Persamaan ini mengkomposisikan orientasi *region* tujuan terhadap orientasi robot saat ini, sehingga robot memiliki arah yang konsisten terhadap *global frame of reference* setelah berpindah region.

3.3.4 Global Planner



Gambar 3.15: Diagram alir proses global planner

Global planner bertugas merencanakan jalur dari posisi robot menuju tujuan dalam skala luas menggunakan peta global. Algoritma yang digunakan adalah A* dengan *wavefront expansion* pada *potential field* yang dihitung dari *costmap* global. Proses *planner* seperti ditunjukkan pada gambar 3.15 dimulai dengan kalkulasi biaya traversal, dilanjutkan dengan ekspansi potensial, ekstraksi gradien, dan pembentukan jalur akhir.

3.3.4.1 Kalkulasi Cost

Kalkulasi *cost* di *global planner* adalah proses transformasi nilai *costmap* global (0–255) menjadi *effective traversal cost* yang digunakan dalam ekspansi *wavefront* (A*):

$$c_{\text{eff}} = \text{costmap}[n] \cdot f + c_{\text{neutral}} \quad (3.33)$$

dengan batasan berikut:

$$c_{\text{eff}} = \begin{cases} c_{\text{eff}} & \text{if } c_{\text{eff}} < c_{\text{lethal}} \\ c_{\text{lethal}} - 1 & \text{if } c_{\text{eff}} \geq c_{\text{lethal}} \\ c_{\text{lethal}} & \text{if } \text{costmap}[n] \geq c_{\text{lethal}} \text{ dan tidak unknown} \end{cases} \quad (3.34)$$

3.3.4.2 Kalkulasi Potensial Kuadratik

Kalkulasi potensial kuadratik adalah aproksimasi Euclidean wavefront yang lebih akurat. Diberikan dua neighbor terkecil dari arah horizontal dan vertikal:

$$t_c = \min(P_{\text{left}}, P_{\text{right}}), \quad t_a = \min(P_{\text{up}}, P_{\text{down}}) \quad (3.35)$$

Selisih: $\Delta = |t_c - t_a|$ (jika $t_c < t_a$, tukar).

Jika $\Delta \geq c_{\text{eff}}$:

$$P(n) = t_a + c_{\text{eff}} \quad (3.36)$$

Jika $\Delta < c_{\text{eff}}$ (interpolasi kuadratik):

$$d = \frac{\Delta}{c_{\text{eff}}} \quad (3.37)$$

$$v = -0.2301 \cdot d^2 + 0.5307 \cdot d + 0.7040 \quad (3.38)$$

$$P(n) = t_a + c_{\text{eff}} \cdot v \quad (3.39)$$

Koefisien polinomial tersebut adalah hasil *least-squares fit* dari fungsi $\sqrt{1 + d^2}$ (jarak Euclidean sebenarnya dari *point source*). Kurva ini mendekati:

$$v(d) \approx \sqrt{1 + d^2} \quad (3.40)$$

dengan error minimal.

3.3.4.3 Algoritma A*

Setelah *potential field* selesai dihitung, algoritma A* melakukan ekspansi dari sel tujuan menuju posisi robot (berlawanan dengan arah navigasi). Prioritas ekspansi setiap sel n ditentukan oleh *key*:

$$\text{key}(n) = P_{\text{quad}}(n) + h(n) \quad (3.41)$$

$P_{\text{quad}}(n)$ adalah potensial dari sel n yang dihitung oleh kalkulator kuadratik melalui interpolasi dua tetangga orthogonal terdekat secara simultan — nilainya mencerminkan jarak dari sel n menuju posisi awal ekspansi (tujuan). Heuristik $h(n)$ adalah estimasi jarak Manhattan dari sel n ke tujuan:

$$h(n) = \text{ManhattanDistance}(n, \text{goal}) \cdot c_{\text{neutral}} \quad (3.42)$$

$$\text{ManhattanDistance}(x, y) = |x_{\text{goal}} - x| + |y_{\text{goal}} - y| \quad (3.43)$$

Karena $P_{\text{quad}}(n)$ dan $h(n)$ sama-sama mengestimasi jarak ke tujuan, heuristik Manhattan tidak memberikan bias ekspansi yang substansial. Pola ekspansi A* pada dasarnya identik dengan Dijkstra karena tidak ada preferensi arah yang signifikan. Dengan demikian, keunggulan utama konfigurasi A* dengan kalkulator kuadratik bukan terletak pada aspek A* nya, melainkan pada kalkulator kuadratik yang menghasilkan *potential field* lebih akurat dan memerlukan lebih sedikit iterasi dibandingkan kalkulasi linear dari satu tetangga.

3.3.4.4 Pembersihan Endpoint

Setelah *potential field* selesai, area 2×2 sel di sekitar tujuan dibersihkan. Langkah ini memastikan bahwa *gradient path* dapat menemukan jalan menuju area tujuan meskipun tujuan berada tepat pada sel *obstacle*. Tanpa pembersihan ini, gradien di sekitar tujuan akan terjebak pada nilai potensial yang tinggi.

3.3.4.5 Ekstraksi Gradient Path

Gradient path bertujuan menghasilkan path yang halus dengan interpolasi sub-sel. Proses ini menggunakan *step* sebesar 0,5 sel per langkah. Komputasi gradien dilakukan dengan aproksimasi *central difference*:

$$g_x = \frac{P(x-1, y) - P(x+1, y)}{2} \quad (3.44)$$

$$g_y = \frac{P(x, y-1) - P(x, y+1)}{2} \quad (3.45)$$

Normalisasi:

$$\hat{g}_x = \frac{g_x}{\sqrt{g_x^2 + g_y^2}}, \quad \hat{g}_y = \frac{g_y}{\sqrt{g_x^2 + g_y^2}} \quad (3.46)$$

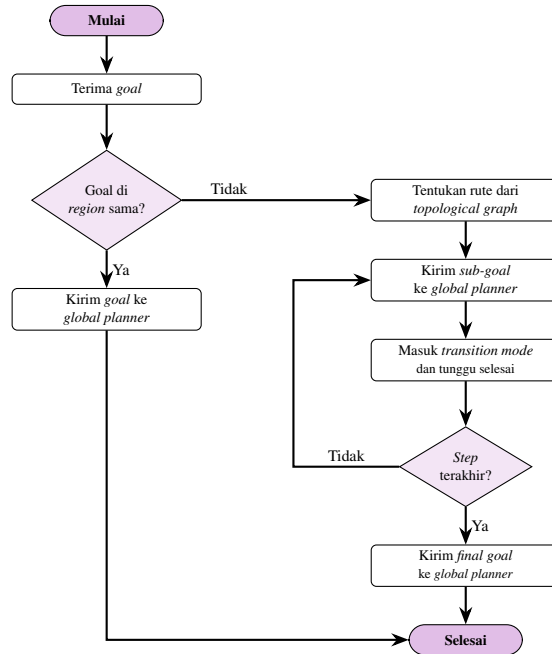
Untuk sel yang berada di *obstacle* (potensial tinggi), gradien dihitung dari tetangga terdekat yang valid. Perhitungan *path step* untuk langkah gradien:

$$\text{step} = \frac{\text{pathStep}}{\sqrt{g_x(p)^2 + g_y(p)^2}} \quad (3.47)$$

$$d_x \leftarrow d_x + g_x(p) \cdot \text{step}, \quad d_y \leftarrow d_y + g_y(p) \cdot \text{step} \quad (3.48)$$

Ketika $|d_x| \geq 1.0$ atau $|d_y| \geq 1.0$, terjadi *cell overflow* dan posisi berpindah ke sel tetangga.

3.3.5 Path Router



Gambar 3.16: Diagram alir proses path router

Path router adalah komponen yang memungkinkan *global planner* bekerja pada navigasi multi-region. Ketika tujuan berada di *region* yang berbeda dari posisi robot saat ini, *path router* menyusun rencana perjalanan bertahap melalui *node* transisi pada *topological graph*. Alur kerja lengkap ditunjukkan oleh gambar 3.16

Pada graph $G = (V, E)$ (Subbab 3.2.3), perjalanan dari suatu *node* awal ke *node* tujuan dilakukan dengan menelusuri *edge* secara berurutan. Sebagai contoh, perjalanan dari v_1 (Entry_{AB}) ke v_4 (Exit_{BC}) dilalui melalui v_2 dan v_3 :

$$P = (v_1, v_2, v_3, v_4) \quad (3.49)$$

Setiap *edge* inter-region (v_1, v_2) dan (v_3, v_4) memerlukan mekanisme lompatan yang ditangani oleh *region switcher*.

Langkah-langkah yang dilakukan *path router* adalah:

1. **Menentukan rute**

Berdasarkan posisi robot dan tujuan, *path router* menentukan urutan *region* yang harus dilalui (misal $A \rightarrow B \rightarrow C$) dan *node* transisi yang relevan.

2. **Mengirim goal bertahap**

Goal dikirim ke setiap *transition point* secara berurutan, dimulai dari *node* transisi pertama.

3. **Menunggu lompatan**

Setelah robot mencapai *node* entry dan *region switcher* melakukan lompatan, *path router* menunggu sinyal transisi selesai sebelum melanjutkan.

4. Mengirim final goal

Setelah semua transisi selesai, *path router* mengirim tujuan akhir ke *global planner*.

Selama proses berjalan, *path router* terus melacak posisi robot dan memperbarui *region* aktif berdasarkan data dari lokalisasi ICP.

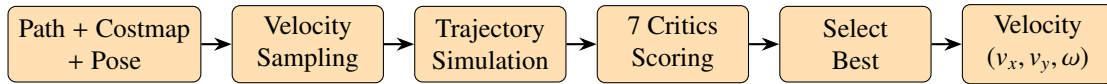
3.3.5.1 Centroid Transisi sebagai Waypoint

Transition costmap mengandung sel transisi ($T = 1$) yang menandai garis batas antar region di dalam peta. Region switcher melakukan pemindaian *ROI window* untuk menemukan sel-sel dengan nilai $T = 1$ dan menghitung *centroid*-nya:

$$\mathbf{c}_T = \frac{1}{N_T} \sum_{i=1}^{N_T} (x_i, y_i) \quad (3.50)$$

Centroid T ini dipublikasikan ke *topic* exttt/transition.centroids dan digunakan oleh *path router* sebagai *waypoint* navigasi menuju area transisi. Ketika *path router* menyusun rute multi-region, ia mengirim *goal* ke centroid T terlebih dahulu sebelum menunggu lompatan oleh *region switcher*.

3.3.6 Dynamic Window Approach (DWA) Local Planner



Gambar 3.17: Diagram alir proses DWA local planner

Dynamic Window Approach (DWA) adalah algoritma *local planner* yang merencanakan pergerakan robot dalam jangka pendek. Pada setiap siklus kontrol, DWA mengambil sampel kecepatan yang mungkin dicapai dalam batas akselerasi robot, mensimulasikan lintasan yang dihasilkan, dan memilih lintasan dengan skor terbaik. Seperti ditunjukkan pada gambar 3.17, algoritma ini bekerja dalam lima langkah utama: (1) sampling kecepatan dalam *dynamic window*, (2) simulasi lintasan menggunakan model kinematik, (3) penilaian lintasan berdasarkan kriteria jarak dan keselarasan, (4) pemilihan lintasan terbaik, dan (5) eksekusi perintah kecepatan.

3.3.6.1 Velocity Window

Velocity window membatasi kecepatan yang dapat dicapai robot dalam satu siklus kontrol berdasarkan akselerasi maksimum. Batas kecepatan minimum dan maksimum untuk setiap sumbu dihitung dengan:

$$v_{\min} = \max(v_{\min_limit}, v_{\text{current}} - \dot{v}_{\max} \cdot \Delta t) \quad (3.51)$$

$$v_{\max} = \min(v_{\max_limit}, v_{\text{current}} + \dot{v}_{\max} \cdot \Delta t) \quad (3.52)$$

dengan Δt adalah durasi satu siklus kontrol. Prinsip yang sama diterapkan pada kecepatan rotasi ω .

3.3.6.2 Velocity Sampling

Setelah batas kecepatan ditentukan, rentang kecepatan pada setiap dimensi (v_x, v_y, ω) dibagi menjadi sampel yang merata:

$$\Delta v_x = \frac{v_{x,\max} - v_{x,\min}}{N_{v_x} - 1} \quad (3.53)$$

$$\Delta v_y = \frac{v_{y,\max} - v_{y,\min}}{N_{v_y} - 1} \quad (3.54)$$

$$\Delta \omega = \frac{\omega_{\max} - \omega_{\min}}{N_{\omega} - 1} \quad (3.55)$$

Total jumlah lintasan yang dievaluasi dalam satu siklus adalah:

$$N_{\text{total}} = N_{v_x} \times N_{v_y} \times N_{\omega} \quad (3.56)$$

Konfigurasi yang digunakan adalah $N_{v_x} = 6$, $N_{v_y} = 6$, $N_{\omega} = 30$, sehingga total 1080 lintasan per evaluasi.

3.3.6.3 Model Kinematik

Setiap sampel kecepatan (v_x, v_y, ω) disimulasikan menjadi lintasan menggunakan model kinematik *differential drive*:

$$x_{t+1} = x_t + \left(v_x \cos \theta_t + v_y \cos \left(\frac{\pi}{2} + \theta_t \right) \right) \cdot \Delta t \quad (3.57)$$

$$y_{t+1} = y_t + \left(v_x \sin \theta_t + v_y \sin \left(\frac{\pi}{2} + \theta_t \right) \right) \cdot \Delta t \quad (3.58)$$

$$\theta_{t+1} = \theta_t + \omega \cdot \Delta t \quad (3.59)$$

Jumlah langkah simulasi ditentukan oleh *granularity*:

$$N_{\text{steps}} = \frac{T_{\text{sim}}}{\Delta t} \quad (3.60)$$

$$\Delta t = \min \left(\frac{\text{sim_granularity}}{|v|}, \frac{\text{angular_sim_granularity}}{|\omega|} \right) \quad (3.61)$$

3.3.6.4 Penilaian Lintasan

Setelah lintasan disimulasikan, setiap lintasan diberi skor berdasarkan beberapa kriteria. Skor total dihitung dengan menjumlahkan kontribusi dari masing-masing *critic*:

$$\begin{aligned}
C_{\text{total}} = & C_{\text{osc}} \\
& + w_{\text{obs}} \cdot C_{\text{obs}} + w_{\text{gf}} \cdot C_{\text{gf}} + w_{\text{align}} \cdot C_{\text{align}} \\
& + w_{\text{path}} \cdot C_{\text{path}} + w_{\text{goal}} \cdot C_{\text{goal}} + w_{\text{twirl}} \cdot C_{\text{twirl}}
\end{aligned} \tag{3.62}$$

Terdapat tujuh *critic* yang digunakan, masing-masing dengan fungsi penilaian yang berbeda:

Tabel 3.7: Daftar kriteria penilaian lintasan

No	Critic	Fungsi
0	Oscillation costs	Mencegah osilasi gerak
1	Obstacle costs	Menghindari tabrakan dengan rintangan
2	Goal front costs	Menilai apakah hidung robot mengarah ke tujuan
3	Alignment costs	Menilai apakah hidung robot sejajar lintasan
4	Path costs	Menilai sejauh mana robot mengikuti lintasan global
5	Goal costs	Menilai jarak robot ke tujuan
6	Twirling costs	Memberi penalti pada rotasi berlebihan

Obstacle cost C_{obs} dihitung berdasarkan sel-sel *costmap* yang dilintasi oleh *footprint* robot. Untuk setiap titik sepanjang lintasan, *footprint* robot di-*rasterize* ke grid *costmap* menggunakan algoritma Bresenham. Jika terdapat sel dengan nilai lethal obstacle (254), lintasan dianggap tidak valid. Nilai C_{obs} dihitung dengan:

$$C_{\text{obs}} = \begin{cases} \sum_{k=1}^N C_k & \text{jika mode penjumlahan} \\ \max_k C_k & \text{sebaliknya} \end{cases} \tag{3.63}$$

3.3.6.5 Kalkulasi Biaya

Penilaian jarak terhadap lintasan global dan tujuan menggunakan struktur data map yang menyebarkan nilai jarak dari sel-sel target menggunakan BFS. Proses propagasi dilakukan dengan langkah-langkah berikut:

1. Setiap sel diinisialisasi dengan nilai *obstacle cost*.
2. Sel target (*waypoint* lintasan global atau titik tujuan) diberi jarak 0 sebagai *seed*.
3. BFS menyebar ke setiap tetangga: sel yang bukan *obstacle* menerima jarak $d_{\text{parent}} + 1$.
4. Propagasi berhenti jika bertemu lethal obstacle atau no information.

Hasil BFS berupa peta jarak:

$$d_{\text{cell}} = \text{jumlah langkah BFS dari sel target terdekat} \tag{3.64}$$

Untuk kriteria yang memerlukan penilaian arah (*goal front* dan *alignment*), titik penilaian digeser ke depan sejauh d_{forward} menggunakan *forward point shift*:

$$p_{\text{score}} = p_{\text{robot}} + d_{\text{forward}} \cdot \begin{pmatrix} \cos \theta \\ \sin \theta \end{pmatrix} \quad (3.65)$$

Path costs (C_{path}) menilai jarak lintasan yang disimulasikan terhadap lintasan global. BFS dilakukan dengan semua *waypoint* dalam jangkauan sebagai *seed*. Bobot:

$$w_{\text{path}} = \text{resolution} \cdot \text{path_distance_bias} \quad (3.66)$$

Goal costs (C_{goal}) menilai jarak ke titik tujuan akhir. BFS hanya menggunakan titik tujuan yang terjangkau sebagai *seed*. Bobot:

$$w_{\text{goal}} = \text{resolution} \cdot \text{goal_distance_bias} \quad (3.67)$$

Goal front costs (C_{gf}) menilai apakah bagian depan robot mengarah ke tujuan. Menggunakan *forward point shift*:

$$p_{\text{hidung}} = p_{\text{robot}} + d_{\text{forward}} \cdot \hat{v} \quad (3.68)$$

$$C_{\text{gf}} = \text{target_dist}(p_{\text{hidung}}) \quad (3.69)$$

Alignment costs (C_{align}) menilai apakah bagian depan robot sejajar dengan lintasan global. Sama dengan *path costs* tetapi dengan *forward point shift*. Dinonaktifkan jika robot sudah dekat dengan tujuan:

$$\text{dinonaktifkan jika } \|p_{\text{robot}} - p_{\text{goal}}\| < d_{\text{forward}} \cdot \text{cheat_factor} \quad (3.70)$$

3.3.6.6 Seleksi Lintasan

Setelah semua lintasan dinilai, lintasan dengan skor total terendah dipilih sebagai perintah kecepatan yang akan dikirim ke robot. Proses seleksi dilakukan sebagai berikut:

Algorithm 4 Seleksi lintasan terbaik

```

1: Siapkan semua critics melalui MapGridCostFunction::prepare()
2: for setiap sampel kecepatan dari generator do
3:   Hasilkan lintasan: Trajectory(pos, vel, sample)
4:   Hitung skor: scoreTrajectory(traj, best_cost)
5:   if cost ≥ 0 dan cost < best_cost then
6:     Simpan sebagai lintasan terbaik
7:   end if
8: end for
9: if tidak ada lintasan valid then
10:   Gunakan kecepatan nol (zero velocity)
11: end if

```

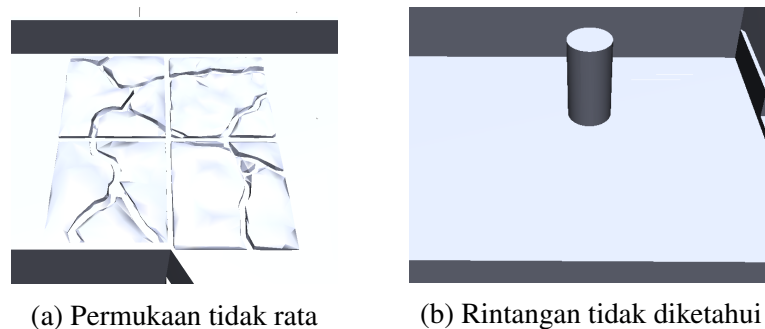
Early termination: Jika total cost sudah melebihi biaya terbaik saat menjumlahkan *critics*, evaluasi lintasan dihentikan lebih awal untuk menghemat komputasi.

3.4 Metode Pengujian dan Evaluasi

Pengujian sistem navigasi berbasis *layered costmap* dilakukan pada lingkungan simulasi Mujoco yang diadaptasi dari arena Kontes Robot SAR Indonesia (KRSRI) 2024. Tujuan pengujian adalah untuk memvalidasi kinerja setiap komponen sistem navigasi, yaitu lokalisasi ICP, *global planner* A*, dan *local planner* DWA, dalam skenario yang merepresentasikan kondisi pasca gempa. Pengujian dilakukan pada tiga region (A, B, C) dengan variasi kondisi dan konfigurasi parameter untuk mengevaluasi efektivitas dan keterbatasan pendekatan yang diusulkan.

3.4.1 Definisi Kondisi Pasca Gempa

Kondisi pasca gempa dalam pengujian ini direpresentasikan melalui dua elemen utama, yaitu rintangan tidak diketahui dan permukaan tidak rata. Rintangan tidak diketahui merujuk pada objek atau puing yang keberadaannya tidak terdeteksi pada peta awal, seperti ditunjukkan pada Gambar 3.18b. Sementara itu, permukaan tidak rata merepresentasikan lantai yang rusak akibat gempa yang menantang kestabilan robot, seperti pada Gambar 3.18a. Kombinasi kedua elemen tersebut membentuk skenario yang menyerupai lingkungan pasca gempa, sehingga dapat digunakan untuk mengevaluasi sistem navigasi yang diusulkan.



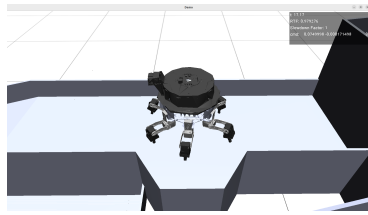
Gambar 3.18: Elemen kondisi pasca gempa

3.4.2 Pengujian Lokalisasi ICP

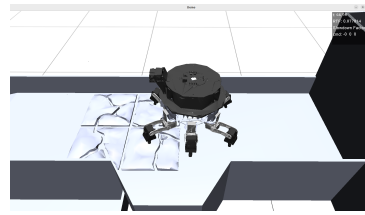
Pengujian lokalisasi ICP bertujuan untuk mengukur akurasi estimasi pose robot pada berbagai kondisi lantai dan region. Kecepatan gerak robot selama pengujian lokalisasi ditetapkan sebesar 0,025 m/s agar diperoleh data *scan* yang cukup padat untuk proses *scan matching*. Metrik yang dievaluasi meliputi error estimasi pose pada sumbu x , sumbu y , dan θ (*yaw*) di sepanjang lintasan robot. Tabel 3.8 menunjukkan nilai parameter ICP yang digunakan saat pengujian. Pengujian dilakukan pada tiga kondisi, yaitu (1) kondisi region A normadatarl (gambar 3.19a), (2) kondisi region A dengan rintangan lantai (gambar 3.19b), dan (3) kondisi region C dengan rintangan tidak diketahui (gambar 3.19c).

Tabel 3.8: Parameter ICP

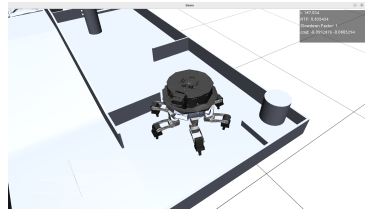
Parameter	Nilai Default	Keterangan
max_iter	50	Iterasi maksimum ICP
s	2	<i>Random stride</i> subsampling
t_{norm}	0.01 m	Batas konvergensi translasi
r_{norm}	0.01 rad ($\approx 0.57^\circ$)	Batas konvergensi rotasi
k	3	Jumlah <i>nearest neighbors</i>
r	10	Radius pencarian <i>neighbor</i>



(a) Simulasi di Region A



(b) Simulasi di Region A dengan rintangan lantai



(c) Simulasi di Region C

Gambar 3.19: Visualisasi lingkungan simulasi di berbagai region

3.4.3 Pengujian Global Planner (A*)

Pengujian *global planner* bertujuan untuk mengevaluasi pengaruh *Cost Scaling Factor* (CSF) terhadap *path* yang dihasilkan oleh algoritma A*. CSF memengaruhi kemiringan gradien biaya inflasi di sekitar *obstacle* sebagaimana dijelaskan pada persamaan fungsi inflasi. Global planner menggunakan *cost function* dengan parameter berikut.

Tabel 3.9: Parameter Default Cost Function

Parameter	Nilai Default	Keterangan
f	0.55	<i>Cost factor</i>
c_{neutral}	1	<i>Neutral cost</i>
c_{lethal}	253	<i>Lethal cost</i>

Pengujian dilakukan pada seluruh tiga region (A, B, C) dengan masing-masing empat variasi nilai *cost scaling factor* (CSF) pada global costmap: 10, 20, 50, dan 100. Metrik evaluasi yang digunakan meliputi:

1. Panjang *path* yang dihasilkan (m).
2. Waktu komputasi yang diperlukan untuk perencanaan *path* (ms).
3. *Minimum clearance* terhadap *obstacle* terdekat (m).
4. Jumlah *waypoint* penyusun *path*.
5. *Cross track error* terhadap *ground truth*, mencakup nilai rata-rata (*mean*), maksimum (*max*), dan simpangan baku (*standard deviation*) (m).

Hasil dari keempat nilai CSF pada setiap region dibandingkan untuk menentukan pengaruh CSF terhadap karakteristik *path* dan menentukan nilai CSF yang paling sesuai untuk masing-masing region. Standar deviasi tiap *path* juga dibandingkan dengan sebuah *ground truth* yang merupakan perkiraan jalur awal terbaik.

3.4.4 Pengujian Local Planner (DWA)

Pengujian *local planner* DWA bertujuan untuk mengevaluasi kemampuan algoritma DWA dalam mengikuti *global path* yang telah direncanakan sebelumnya. Parameter pada setiap *region* dikonfigurasi secara berbeda sesuai dengan karakteristik medan:

Tabel 3.10: Konfigurasi parameter DWA setiap region

Parameter	A	B	C
max v_x	0,08	0,04	0,05
min v_x	0,05	0,02	-0,05
max v_y	0,08	0,04	0,05
min v_y	-0,08	-0,04	-0,05
sim_time	5,0	5,0	10,0
v_x samples	6	6	18
v_y samples	6	6	18
v_θ samples	30	30	30
path bias	16,0	16,0	16,0
goal bias	32,0	32,0	24,0

Pada Region A dan B yang memiliki area lebih luas, robot dapat bergerak dengan kecepatan yang lebih tinggi. Sementara itu, Region C yang memiliki area lebih sempit dan *trajectory* yang lebih kompleks dibatasi dengan kecepatan yang lebih rendah untuk menjaga stabilitas navigasi. Metrik yang digunakan adalah waktu tempuh, serta *mean cross track error* dan *max cross track error* terhadap *global path*.

3.5 Urutan Pelaksanaan Penelitian

Penelitian ini dilaksanakan dalam tujuh tahap utama sebagai berikut:

1. Studi Literatur

Tahap awal ini akan fokus pada pengumpulan dan pemahaman mendalam teori dasar *layered costmap*. Selain itu, akan dipelajari berbagai algoritma navigasi, seperti A*, yang relevan untuk *path planning* robot, serta pemahaman mengenai teknik lokalisasi, seperti

ICP. Tujuannya adalah untuk membangun dasar teoritis yang kuat sebagai landasan perancangan dan implementasi sistem navigasi.

2. Perancangan Sistem

Pada tahap ini, struktur *layered costmap* akan didesain secara rinci, termasuk penentuan jumlah lapisan dan jenis informasi yang akan dimuat pada setiap lapisan. Alur data dari berbagai sensor akan dirancang untuk memastikan informasi lingkungan dapat diintegrasikan dengan efisien ke dalam *costmap*. Arsitektur sistem navigasi secara keseluruhan akan dibentuk, mempertimbangkan interaksi antara modul sensor LiDAR, lokalisasi, perencanaan jalur, dan kontrol gerak robot.

3. Perancangan Lingkungan Simulasi

Lingkungan simulasi akan dirancang untuk merepresentasikan kondisi pasca gempa dalam skala laboratorium. Arena simulasi diadaptasi dari arena Kontes Robot SAR Indonesia (KRSRI) 2024 yang mencakup beberapa elevasi, tanjakan, dan permukaan tidak rata. Lingkungan ini dimodelkan di dalam simulator MuJoCo untuk keperluan pengujian sistem navigasi.

4. Implementasi Navigasi

Modul *layered costmap* yang telah dirancang akan diimplementasikan ke dalam sistem perangkat lunak robot, memastikan integrasi yang mulus dengan data sensor. Algoritma lokalisasi akan dikodekan dan disesuaikan untuk bekerja secara efektif dengan *layered costmap*, memungkinkan robot mengetahui posisinya secara presisi di berbagai level. Terakhir, algoritma navigasi yang dipilih akan diintegrasikan untuk memungkinkan robot merencanakan dan mengeksekusi pergerakan antar titik tujuan, termasuk transisi antar elevasi.

5. Uji Coba dan Evaluasi

Pengujian akan dilakukan untuk menguji kemampuan robot dalam bernavigasi antar titik tujuan yang terletak pada level berbeda di lingkungan simulasi. Pengujian ini akan mencakup skenario yang bervariasi, seperti navigasi melalui rintangan, menaiki dan menuruni elevasi, dan mencapai tujuan di region yang berbeda. Performa robot akan dievaluasi berdasarkan metrik seperti akurasi pencapaian tujuan, waktu tempuh, dan kelancaran pergerakan.

6. Analisis Hasil

Data yang terkumpul dari tahap uji coba akan dianalisis secara mendalam untuk mengidentifikasi kekuatan dan kelemahan dari pendekatan navigasi yang menggunakan *layered costmap*. Analisis juga akan mencakup identifikasi faktor-faktor yang mempengaruhi performa robot dan potensi perbaikan di masa mendatang.

7. Penyusunan Laporan

Seluruh hasil penelitian, mulai dari studi literatur, perancangan sistem, implementasi, hingga analisis hasil uji coba, akan didokumentasikan secara lengkap dalam bentuk laporan. Laporan ini akan disusun sesuai dengan standar penulisan ilmiah sebagai tugas akhir.

Tabel 3.11: Tabel timeline

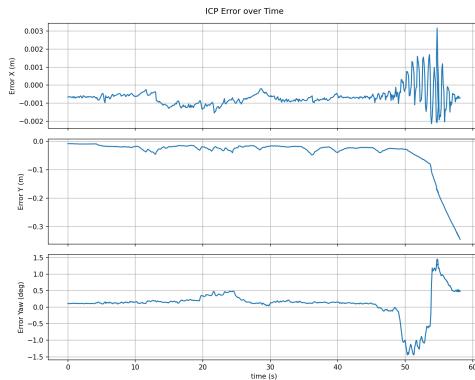
Kegiatan	Minggu															
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Studi Literatur																
Perancangan Sistem																
Perancangan Lingkungan Simulasi																
Implementasi Navigasi																
Uji Coba dan Evaluasi																
Analisis Hasil																
Penyusunan Laporan																

[Halaman ini sengaja dikosongkan]

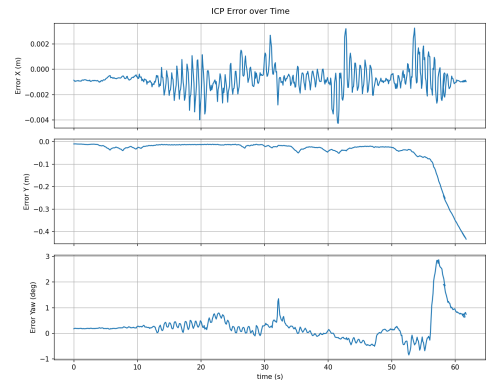
BAB IV

HASIL DAN PEMBAHASAN

4.1 Hasil dan Analisis Lokalisasi ICP



(a) Error ICP di Region A

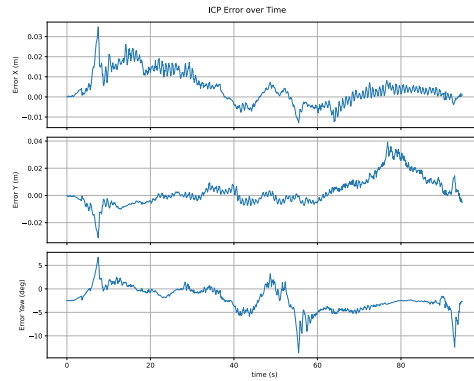


(b) Error ICP di Region A pada lantai tidak rata

Gambar 4.1: Perbandingan error ICP di Region A

Grafik pada Gambar 4.1a menunjukkan error ICP pada sumbu x , y , dan θ (yaw) saat robot melintasi Region A. Secara umum, performa ICP stabil dengan tingkat error yang rendah sepanjang lintasan. Namun, terjadi lonjakan error yang signifikan pada sekitar $t = 50$ detik. Lonjakan ini disebabkan oleh posisi robot yang berada di zona transisi antara dua *region*, di mana lingkungan yang dapat terdeteksi oleh sensor LiDAR sangat terbatas. Akibatnya, kualitas *scan matching* menurun dan menghasilkan akumulasi error yang lebih besar pada estimasi transformasi.

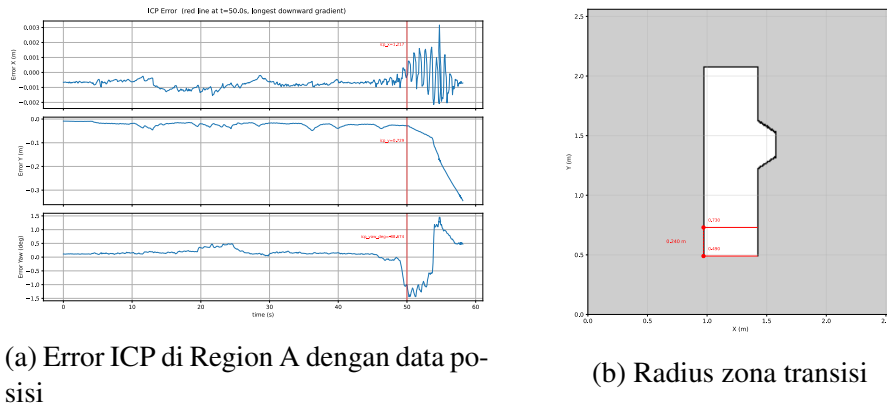
Selanjutnya, performa ICP diuji pada region A dengan kondisi lantai yang tidak rata untuk mensimulasikan skenario lantai yang rusak akibat gempa bumi. Gambar 4.1b menunjukkan error ICP pada sumbu x , y , dan θ (yaw) untuk skenario ini. Dibandingkan dengan kasus sebelumnya, terjadi peningkatan osilasi yang signifikan, terutama pada sumbu x dan yaw. Hal ini disebabkan oleh permukaan lantai yang tidak rata sehingga menghasilkan titik-titik *scan* LiDAR yang lebih bervariasi secara vertikal, yang menurunkan kualitas *scan matching* pada proses ICP. Akibatnya, estimasi transformasi menjadi kurang stabil dan menghasilkan fluktuasi error yang lebih besar. Lonjakan error di sekitar $t = 55$ detik tetap terjadi karena faktor zona transisi antar *region*.



Gambar 4.2: Error ICP di Region C

Selanjutnya, performa ICP diuji pada Region C. Gambar 4.2 menunjukkan error ICP pada sumbu x , y , dan θ (yaw) untuk skenario ini. Hasilnya menunjukkan bahwa error maksimum pada ketiga sumbu lebih tinggi dibandingkan dengan performa di Region A, dan secara umum error yang dihasilkan lebih tidak stabil. Hal ini disebabkan oleh dua faktor. Pertama, Region C memiliki jumlah sel *obstacle* yang lebih banyak, sehingga tabel *hash* KNN yang perlu dicari lebih besar dan berpotensi memperlambat serta menurunkan kualitas *scan matching*. Kedua, *trajectory* robot di Region C lebih kompleks dengan banyak rotasi, yang mengakibatkan akumulasi error yang lebih besar pada estimasi *yaw*.

4.1.1 Hasil Zona Transisi

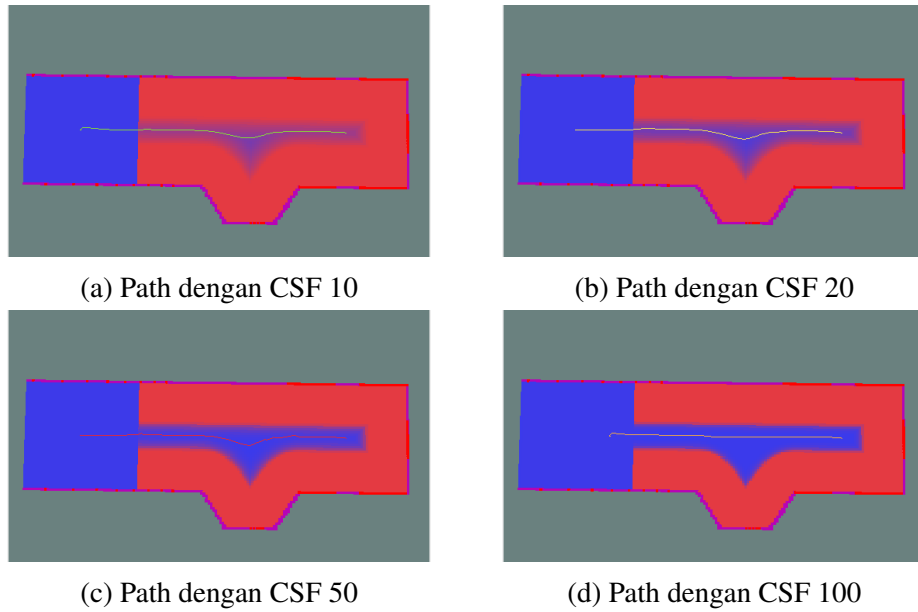


Gambar 4.3: Visualisasi error ICP dan radius zona transisi

Berdasarkan Gambar 4.1a, dapat pula diidentifikasi posisi robot saat kualitas ICP mulai menurun. Dari data 4.3a, penurunan kualitas ICP mulai terjadi ketika robot berada pada posisi $y = 0.729$ m, atau sekitar 0.24 m dari ujung *map*. Gambar 4.3b menunjukkan bahwa zona transisi perlu dirancang dengan radius minimal 0.24 m agar robot dapat melakukan transisi antar *region* dengan aman sebelum kualitas lokalisasi mengalami penurunan signifikan.

4.2 Hasil dan Analisis Global Planner

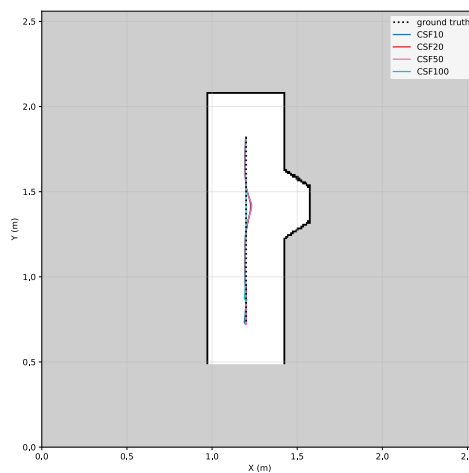
4.2.1 Hasil Path di Region A



Gambar 4.4: Relasi path terhadap Cost Scaling Factor di Region A

Cost Scaling Factor	Panjang path (m)	Waktu komputasi (ms)	Min clearance (m)	Jumlah waypoint
10	1.116051	6.666027	0.220000	222
20	1.114836	6.897399	0.220000	223
50	1.122597	5.688345	0.219650	225
100	1.071836	6.484874	0.216470	193

Tabel 4.1: Karakteristik Path di Region A berdasarkan Cost Scaling Factor

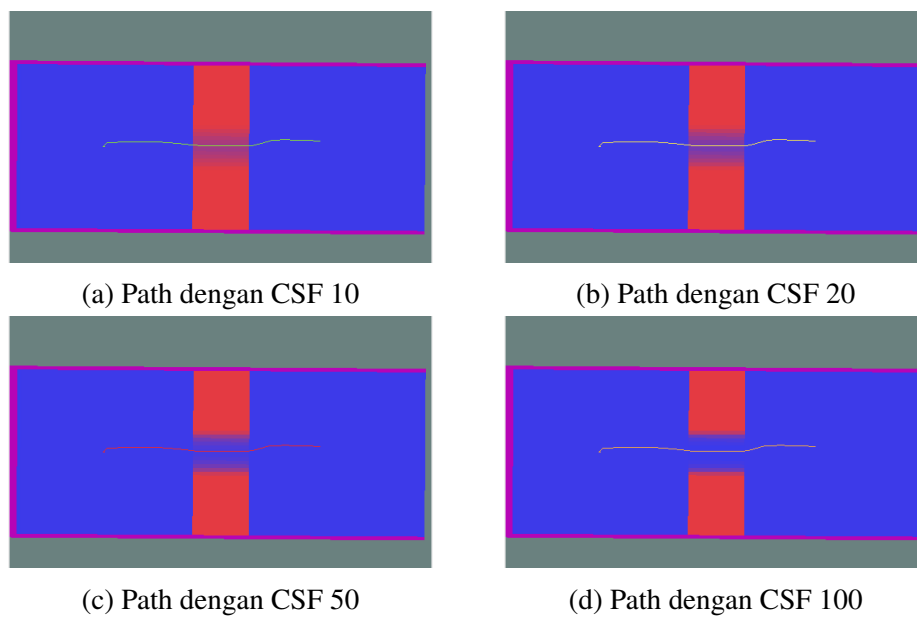


Gambar 4.5: Semua path di region A.

Global path adalah jalur yang direncanakan dari posisi robot menuju suatu tujuan. Pada pengujian di Region A, tujuan yang diberikan adalah titik tengah zona transisi yang ditandai dengan area biru di sebelah kiri peta, yang berfungsi sebagai gerbang menuju *region* berikutnya. Pengujian dilakukan dengan berbagai nilai *Cost Scaling Factor* (CSF) untuk melihat pengaruhnya terhadap bentuk *path* yang dihasilkan. Pengaruh CSF terhadap *global costmap* dijelaskan pada persamaan (3.25), yakni semakin tinggi nilai CSF, semakin curam penurunan nilai biaya terhadap jarak dari *obstacle*.

Hasil *path* untuk setiap nilai CSF ditunjukkan pada Gambar 4.4 dan karakteristiknya dirangkum pada Tabel 4.1. Pada CSF 10, *path* cenderung mengikuti titik tengah di antara dua *obstacle*. Hal ini menghasilkan *path* yang tidak optimal karena melakukan belokan yang tidak diperlukan dan memiliki panjang yang bukan terpendek. Sebaliknya, pada CSF 100, *path* bergerak lebih mepet ke *obstacle*, memanfaatkan area biaya rendah yang lebih luas di dekat dinding. Hasilnya adalah *path* yang lebih lurus dan lebih pendek. Perbandingan dekat dapat dilihat dari gambar 4.5.

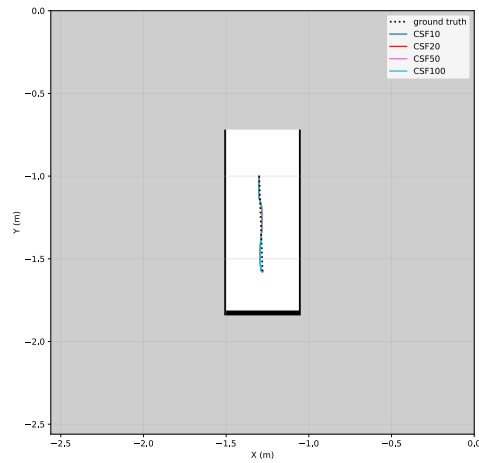
4.2.2 Hasil Path di Region B



Gambar 4.6: Relasi path terhadap Cost Scaling Factor di Region B

Cost Scaling Factor	Panjang path (m)	Waktu komputasi (ms)	Min clearance (m)	Jumlah waypoint
10	0.591730	4.931334	0.208163	118
20	0.591773	4.867434	0.208066	118
50	0.591600	5.123322	0.208224	118
100	0.591590	4.925262	0.208132	118

Tabel 4.2: Karakteristik Path di Region B berdasarkan Cost Scaling Factor



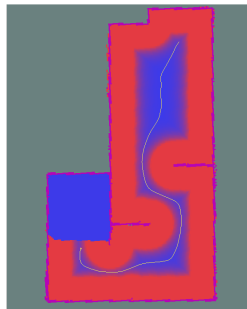
Gambar 4.7: Semua path di region B.

Hasil *path* untuk setiap nilai CSF di Region B ditunjukkan pada Gambar 4.6 dan dilengkapi data kuantitatif pada Tabel 4.2. Seluruh *path* dimulai dari posisi robot setelah bertransisi dari Region A ke Region B dan berakhir di zona transisi menuju Region C. Perbedaan nilai CSF tidak memberikan pengaruh yang signifikan terhadap bentuk *path* yang dihasilkan. Hal ini disebabkan oleh *layout* Region B yang hanya berupa lorong sempit, sehingga tidak terdapat ruang terbuka yang cukup bagi variasi *cost gradient* untuk mengubah lintasan *path* secara berarti. Akibatnya, semua *path* dari keempat nilai CSF hampir identik satu sama lain, sebagaimana terlihat pada Gambar 4.7.

4.2.3 Hasil Path di Region C



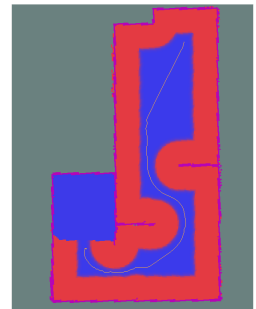
(a) Path dengan CSF 10



(b) Path dengan CSF 20



(c) Path dengan CSF 50

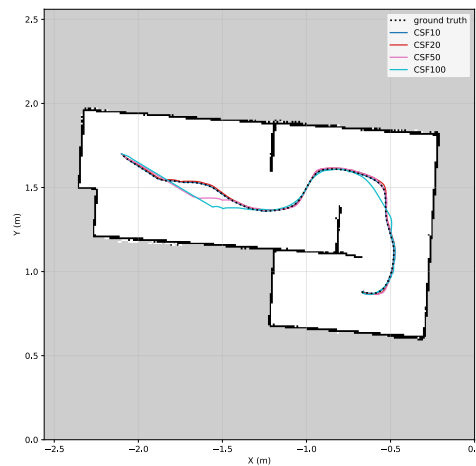


(d) Path dengan CSF 100

Gambar 4.8: Relasi path terhadap Cost Scaling Factor di Region C

Cost Scaling Factor	Panjang path (m)	Waktu komputasi (ms)	Min clearance (m)	Jumlah waypoint
10	2.542122	10.629525	0.197048	509
20	2.596590	10.268101	0.196292	521
50	2.596873	9.922248	0.197185	520
100	2.524643	10.224478	0.195499	506

Tabel 4.3: Karakteristik Path di Region C berdasarkan Cost Scaling Factor



Gambar 4.9: Semua path di region C.

Hasil *path* untuk setiap nilai CSF di Region C ditunjukkan pada Gambar 4.8 dan dilengkapi data kuantitatif pada Tabel 4.3. Pada CSF 10, *path* cenderung mempertahankan posisi di tengah lorong pada semua kondisi, yang menghasilkan jalur yang lebih aman namun lebih panjang. Sebaliknya, pada CSF 100, *path* tidak terikat pada posisi tengah, sehingga mampu memanfaatkan ruang area terbuka, dan menghasilkan *path* yang lebih pendek dan lurus. *Path* pada CSF 20 dan CSF 50 menunjukkan karakteristik peralihan di antara kedua ekstrem tersebut. Dari keseluruhan hasil, CSF 100 menghasilkan *path* yang paling sesuai untuk navigasi di Region C karena memberikan keseimbangan terbaik antara efisiensi jalur (lebih pendek) dan *clearance* terhadap *obstacle* yang masih memadai. Perbandingan dekat dapat dilihat dari gambar 4.9.

4.2.4 Hasil Deviasi Path

Cost Scaling Factor	Mean cross track error (m)	Max cross track error (m)
10	0.006029	0.025937
20	0.006880	0.029719
50	0.006536	0.033099
100	0.003537	0.010000

Tabel 4.4: Perbandingan path terhadap ground truth di region A

Cost Scaling Factor	Mean cross track error (m)	Max cross track error (m)
10	0.005756	0.011590
20	0.005778	0.011591
50	0.005678	0.011591
100	0.005678	0.011591

Tabel 4.5: Perbandingan path terhadap ground truth di region B

Cost Scaling Factor	Mean cross track error (m)	Max cross track error (m)
20	0.005548	0.002228
50	0.012802	0.018897
100	0.020026	0.024742

Tabel 4.6: Perbandingan path terhadap ground truth di region C

Ketiga tabel di atas menunjukkan deviasi *path* yang dihasilkan oleh *global planner* terhadap *ground truth* pada masing-masing *region*. *Ground truth* di sini merupakan *path* rancangan jalur awal, sedangkan *path* hasil perencanaan mengikuti distribusi biaya pada *costmap* yang dipengaruhi oleh nilai CSF

Pada Region A, CSF 10 menghasilkan deviasi yang lebih besar karena *path* cenderung mengikuti jalur tengah di antara *obstacle*, sehingga menyimpang dari *ground truth*. Sebaliknya, CSF 100 menghasilkan *path* yang lebih mendekati *ground truth* dengan deviasi yang lebih rendah pada semua metrik, baik *mean cross track*, *max cross track*, maupun *standard deviation*.

Pada Region B, seluruh nilai CSF menghasilkan *path* yang hampir identik satu sama lain, sehingga deviasi terhadap *ground truth* juga relatif seragam. Hal ini disebabkan oleh *layout* Region B yang hanya berupa lorong sempit, di mana variasi CSF tidak cukup untuk mengubah lintasan *path* secara signifikan.

Pada Region C, CSF 100 menghasilkan *path* yang lebih pendek dan efisien, namun deviasinya terhadap *ground truth* justru paling tinggi dibandingkan nilai CSF lainnya. Hal ini menunjukkan *path* yang lebih pendek belum tentu memiliki deviasi lebih rendah terhadap referensi, karena *ground truth* itu sendiri dirancang berdasarkan jalur yang . Meskipun demikian, dalam konteks navigasi, panjang *path* yang lebih pendek sering kali lebih diutamakan selama deviasi masih dalam batas aman.

4.3 Hasil dan Analisis DWA Local Planner

Percobaan	Mean CTE A (m)	Mean CTE B (m)	Mean CTE C (m)	Max CTE A (m)	Max CTE B (m)	Max CTE C (m)
1	0.009576	0.028042	0.051642	0.022214	0.079888	0.182401
2	0.012385	0.027261	0.058659	0.03001	0.085333	0.175021
3	0.006878	0.032313	0.048686	0.017482	0.101489	0.149008
4	0.012966	0.022426	—	0.025928	0.080275	—
5	0.010471	0.023502	0.053426	0.024355	0.085748	0.150631
6	0.012881	0.02536	0.058164	0.027141	0.084145	0.15028
7	0.004362	0.040043	0.041391	0.01086	0.107007	0.163038
8	0.009971	0.016584	—	0.018826	0.063391	—
9	0.005387	0.037433	0.038019	0.020374	0.099163	0.187151
10	0.007621	0.035032	0.034088	0.018544	0.106812	0.178197
Mean	0.008695	0.031123	0.048009	0.021373	0.093698	0.166966

Tabel 4.7: Rangkuman cross track error per run

Pengujian *DWA planner* bertujuan untuk mengevaluasi kemampuannya dalam mengikuti *global path* yang telah direncanakan sebelumnya. Nilai yang dievaluasi adalah *cross track error* (CTE), yaitu jarak tegak lurus antara posisi robot pada *trajectory* aktual dan titik terdekat pada *global path*. CTE menunjukkan seberapa besar deviasi antara lintasan yang dihasilkan oleh *local planner* terhadap lintasan referensi yang telah direncanakan oleh *global planner*. Pada region A, nilai *mean CTE* sangat rendah (0.0087 m) dengan *max CTE* 0.0214 m, menunjukkan bahwa *DWA* mengikuti *global path* dengan presisi pada lintasan lurus dan sederhana. Pada region B, *mean CTE* sedikit meningkat menjadi 0.0311 m dengan *max CTE* 0.0937 m, yang masih tergolong rendah mengingat lorong sempit pada region ini memberikan ruang bergerak yang lebih terbatas bagi robot.

Pada region C, *max cross track error* meningkat signifikan menjadi 0.1670 m, hampir delapan kali lipat dibandingkan region A. Peningkatan ini disebabkan oleh keberadaan *obstacle* yang tidak tercatat pada *global costmap* (unknown obstacle) dan hanya terdeteksi oleh sensor LiDAR saat navigasi berlangsung. Ketika *obstacle* tersebut berada tepat pada lintasan *global path*, *DWA planner* tidak memiliki pilihan selain menghasilkan *trajectory* alternatif yang menyimpang dari *global path* untuk menghindari tabrakan. Simpangan inilah yang tercermin sebagai lonjakan nilai *max cross track error*. Meskipun demikian, *mean CTE* region C tetap terkendali pada 0.0480 m, menandakan bahwa setelah berhasil melewati *obstacle*, *DWA* kembali mengikuti *global path* dengan baik. Perilaku ini membuktikan bahwa *DWA* mampu menyeimbangkan antara *obstacle avoidance* dan *path following* secara real-time.

4.4 Pembahasan

4.4.1 Pembahasan Costmap

Setiap lapisan *costmap* memiliki parameter yang dapat diatur secara independen sesuai kebutuhan tiap *region*. *Global costmap* dikonfigurasi melalui nilai *Cost Scaling Factor* (CSF) yang memengaruhi gradien biaya inflasi di sekitar *obstacle*. Semakin tinggi CSF, *path* yang dihasilkan semakin pendek karena robot diperbolehkan melintas lebih dekat ke *obstacle*. Namun, nilai CSF yang terlalu tinggi dapat menyebabkan *path* tidak aman jika *clearance* terhadap *obstacle* terlalu kecil. Untuk mengakomodasi perbedaan karakteristik medan, CSF dapat diatur berbeda pada tiap *region*, misalnya CSF tinggi pada area terbuka dan CSF rendah pada lorong

sempit.

Local costmap dapat dikonfigurasi melalui (*window size*) dan resolusi grid. Ukuran jendela yang lebih besar memberikan informasi lingkungan yang lebih luas bagi DWA *local planner* dalam memilih *trajectory*, tetapi memerlukan lebih banyak sumber daya komputasi. Sebaliknya, ukuran jendela yang terlalu kecil dapat menyebabkan DWA tidak memiliki cukup informasi untuk menghindari *obstacle* yang berada di luar jangkauan. Parameter ini perlu disesuaikan dengan kecepatan robot dan kerapatan *obstacle* di lingkungan sekitar.

Transition costmap memiliki parameter radius ekspansi dan nilai *cost* pada zona transisi. Radius ekspansi menentukan seberapa jauh zona buffer menyebar dari garis transisi, yang memengaruhi seberapa awal robot mendeteksi bahwa ia akan berpindah *region*. Nilai *cost* pada zona transisi ($E = 5$) dan garis transisi ($T = 1$) digunakan oleh *region switcher* untuk menentukan kapan robot harus bersiap melakukan lompatan dan kapan lompatan dieksekusi.

Kemampuan untuk mengonfigurasi setiap lapisan *costmap* secara terpisah memberikan fleksibilitas dalam menyesuaikan perilaku navigasi terhadap kondisi lingkungan yang berbeda. Pendekatan ini sejalan dengan prinsip *layered costmap*, di mana setiap lapisan bertanggung jawab atas aspek lingkungan yang spesifik dan dapat dioptimalkan tanpa memengaruhi lapisan lainnya.

4.4.2 Pembahasan Lokalisasi

Sistem lokalisasi merupakan hasil kerja sama antara ICP dan *region switcher*. ICP bertanggung jawab melacak posisi robot secara presisi pada tingkat sub-meter, sedangkan *region switcher* melacak lokasi robot pada tingkat yang lebih besar, yaitu menentukan *region* mana yang sedang ditempati robot. Pembagian peran ini memungkinkan sistem untuk tetap mengetahui posisi robot secara global meskipun ICP mengalami penurunan akurasi di area tertentu, seperti zona transisi antar *region*.

Proses pelompatan *region* menggunakan mekanisme *transition mode* yang memanfaatkan data *pitch* dari sensor kompas CMPS14 untuk mendeteksi perubahan kemiringan saat robot memasuki *slope* pada Region B. Ketika nilai *pitch* melebihi ambang batas (12°), sistem langsung memicu lompatan tanpa harus menunggu *timer* selesai. Mekanisme ini lebih responsif dibandingkan *timer-based* karena mendeteksi transisi secara langsung berdasarkan perubahan fisik lingkungan, bukan berdasarkan estimasi waktu. *Timer-based transition* tetap dipertahankan sebagai *safety fallback* jika data *pitch* tidak tersedia atau tidak mencukupi ambang batas.

4.4.3 Pembahasan Path Planner

Dalam sistem navigasi multi-*region*, *global planner* dan *path router* bekerja sama untuk merencanakan perjalanan dari posisi robot menuju tujuan akhir. Bentuk *path* pada setiap segmen dapat diubah melalui parameter *planner* itu sendiri, tidak hanya dari *costmap*. Parameter seperti *cost factor*, *neutral cost*, dan *Cost Scaling Factor* memengaruhi bagaimana *global planner* menyeimbangkan antara jarak tempuh dan jarak aman terhadap *obstacle*. Setiap kali robot melompat ke *node* baru, *global planner* membuat *path* baru untuk segmen selanjutnya, sehingga *path* yang digunakan selalu sesuai dengan posisi dan *region* robot saat itu.

Hasil pengujian menunjukkan bahwa tidak ada satu nilai CSF yang optimal untuk semua *region*. Pada Region C yang memiliki ruang terbuka, CSF 100 menghasilkan *path* terpendek namun deviasinya terhadap *ground truth* paling tinggi. Sebaliknya, pada Region B yang berupa lorong, seluruh nilai CSF menghasilkan *path* yang hampir identik. Temuan ini menunjukkan

perlunya strategi perencanaan jalur yang bersifat spesifik per *region*. Dengan demikian, setiap segmen dapat menggunakan konfigurasi *global planner* yang berbeda.

4.4.4 Pembahasan DWA Local Planner

Pergerakan robot di lingkungan simulasi menghadapi beberapa lorong sempit, terutama di Region C. Robot hexapod yang digunakan cukup kuat untuk tidak mudah tersangkut, namun radius *footprint* tetap perlu dikonfigurasi dengan tepat agar dapat menghindari tabrakan dengan dinding. Parameter DWA memungkinkan robot bergerak secara *omnidirectional*, yang membantu dalam melewati lorong-lorong sempit karena robot dapat bergerak menyamping tanpa harus berputar terlebih dahulu.

Konfigurasi DWA yang berbeda pada tiap *region* memungkinkan *trajectory* yang dihasilkan tetap optimal sesuai karakteristik medan. Pada Region A yang luas, kecepatan tinggi (0,08 m/s) digunakan untuk meminimalkan waktu tempuh. Pada Region B yang berupa lorong sempit, kecepatan diturunkan (0,04 m/s) untuk menjaga stabilitas navigasi pada ruang terbatas. Pada Region C dengan *obstacle* yang kompleks, jumlah sampel kecepatan ditingkatkan (18 sampel) dan *sim_time* diperpanjang (10 detik) agar DWA memiliki lebih banyak kandidat lintasan untuk menemukan *trajectory* yang aman. Selain itu, *goal bias* pada Region C diturunkan menjadi 24 (dari 32) agar DWA lebih fleksibel dalam memilih lintasan yang menghindari *obstacle* tanpa terlalu terikat pada arah tujuan. Penyesuaian ini menunjukkan bahwa konfigurasi DWA spesifik per *region* berkontribusi langsung terhadap keberhasilan navigasi.

4.4.5 Pembahasan Navigasi Penuh dari Region A ke C

Percobaan	Waktu A (s)	Waktu T0 (s)	Waktu B (s)	Waktu T1 (s)	Waktu C (s)	Total (s)	Sukses
1	13.56	12.52	5.46	7.73	214.54	233.56	Ya
2	14.50	14.33	6.41	7.27	165.45	186.36	Ya
3	14.50	16.96	7.73	6.98	151.12	173.35	Ya
4	15.59	11.75	7.33	7.86	115.32	138.24	Tidak
5	15.23	13.60	7.10	6.50	129.48	151.81	Ya
6	19.00	16.76	7.85	7.97	200.13	226.97	Ya
7	19.00	16.00	8.28	8.24	194.00	221.28	Ya
8	21.39	18.74	15.67	17.69	103.95	141.01	Tidak
9	18.43	16.55	8.77	8.04	166.53	193.74	Ya
10	17.48	14.97	7.83	8.48	182.42	207.73	Ya
Mean	16.46	15.21	7.43	7.65	175.46	199.35	80%

Tabel 4.8: Rangkuman waktu tempuh

Pengujian navigasi penuh dari region A ke region C dilakukan untuk mengevaluasi hasil keseluruhan sistem navigasi. Data waktu tempuh dibagi menjadi waktu di tiap region (A, B, dan C) serta waktu robot di zona transisi T0 (antara A dan B) dan T1 (antara B dan C). Dari 10 kali percobaan, diperoleh 8 keberhasilan dan 2 kegagalan. Waktu tempuh terbaik adalah 141.01 detik dan waktu terlama 233.56 detik, yang menunjukkan variasi yang cukup besar dalam performa navigasi antar percobaan.

Variasi waktu tempuh terbesar terjadi di region C. Hal ini disebabkan oleh karakteristik *DWA planner* yang harus mensimulasikan lintasan dengan *unknown obstacle*. Ketika robot

mendekati *unknown obstacle* yang terdeteksi oleh LiDAR saat navigasi berlangsung, DWA menurunkan kecepatan dan memilih lintasan yang lebih aman, sehingga memperpanjang waktu tempuh. Sebaliknya, pada region A dan B dengan lingkungan yang lebih sederhana dan tanpa *unknown obstacle*, durasi navigasi relatif konsisten antar percobaan. Hal ini terlihat dari nilai Total yang bervariasi antara 141.01 hingga 233.56 detik, di mana variasi tersebut hampir seluruhnya berasal dari fluktuasi waktu di region C (103.95-214.54 detik) akibat perbedaan jumlah dan posisi *unknown obstacle* yang dihadapi robot pada setiap percobaan.

Dua kegagalan terjadi pada percobaan ke-4 dan ke-8. Pada percobaan ke-4, robot gagal menghasilkan lintasan setelah keluar dari *transition mode* di region C. *DWA planner* tidak menemukan *trajectory* yang valid dalam *dynamic window* akibat konfigurasi kecepatan dan orientasi robot yang tidak ideal pasca lompatan, sehingga sistem tidak dapat melanjutkan navigasi. Pada percobaan ke-8, kegagalan disebabkan oleh hilangnya lokalisasi ICP setelah melewati *transition mode*. Koneksi pose yang tidak stabil pasca lompatan menyebabkan *local planner* menerima referensi posisi yang salah, sehingga navigasi tidak dapat dilanjutkan. Kedua kegagalan ini menunjukkan bahwa *transition mode* merupakan titik kritis dalam sistem navigasi multi-region.

Secara keseluruhan, seluruh komponen navigasi (lokalisasi ICP, *global planner A**, *path router*, *region switcher*, dan *DWA local planner*) terintegrasi dengan baik dan mampu melakukan navigasi dari region A ke region C secara otonom. Keberhasilan 8 dari 10 percobaan menunjukkan tingkat keandalan yang memadai, dengan dua kegagalan yang disebabkan oleh kondisi tepi pada *transition mode* yang dapat diminimalkan melalui optimalisasi parameter *transition costmap* dan penanganan *edge case* pada mekanisme lompatan.

[Halaman ini sengaja dikosongkan]

BAB V

PENUTUP

5.1 Kesimpulan

Layered costmap digunakan untuk merepresentasikan lingkungan 3D multi-lantai sebagai kumpulan peta 2D *region* yang saling terhubung. Lingkungan 3D dipisahkan menjadi tiga *region* (A, B, C) yang digabung menjadi satu *combined map* 2D. Setiap *region* memiliki *costmap* global, lokal, dan transisi yang dikonfigurasi secara independen. *Transition map* menyimpan metadata koneksi antar *region* meliputi pasangan koneksi antar *region*, orientasi *quaternion*, dan posisi awal robot. Representasi ini memungkinkan algoritma navigasi 2D bekerja pada lingkungan 3D karena setiap *region* dapat ditangani secara independen menggunakan *costmap* 2D konvensional.

Algoritma lokalisasi ICP menggunakan *hash-based KNN* untuk estimasi pose robot secara presisi. Selama *transition mode*, ICP dinonaktifkan sementara untuk mencegah koreksi pose yang tidak stabil pada zona transisi. *Path router* memecah rute multi *region* menjadi segmen per *region* berdasarkan *topological graph* dan mengirimkan *sub-goal* ke setiap *node* transisi secara berurutan. *Global planner* menggunakan algoritma A* dan parameter *Cost Scaling Factor* (CSF) mengatur keseimbangan antara efisiensi jalur dan keamanan terhadap *obstacle*, dengan CSF 100 menghasilkan *path* terpendek pada semua *region*. *Local planner* DWA dikonfigurasi berbeda pada tiap *region*, dengan kecepatan tinggi di Region A, kecepatan rendah di Region B, serta jumlah sampel dan *sim_time* lebih besar di Region C untuk menghasilkan *trajectory* yang optimal sesuai karakteristik medan masing-masing.

Robot darat otonom berhasil melakukan navigasi dari Region A ke Region C pada lingkungan simulasi Mujoco dengan tingkat keberhasilan 8 dari 10 percobaan. Waktu tempuh rata-rata untuk percobaan sukses adalah 199,35 detik, dengan variasi terbesar terjadi di Region C akibat perbedaan jumlah dan posisi *unknown obstacle* yang dihadapi robot pada setiap percobaan. Dua kegagalan terjadi akibat *transition mode* dimana percobaan 4 gagal karena DWA tidak menghasilkan *trajectory* valid pasca lompatan, dan percobaan 8 gagal karena kehilangan lokalisasi ICP setelah keluar dari *transition mode*. Seluruh komponen navigasi: lokalisasi ICP, *layered costmap*, *global planner* A*, *path router*, *region switcher*, dan *local planner* DWA, terintegrasi dengan baik dan membuktikan bahwa pendekatan navigasi antar lantai berbasis *layered costmap* layak digunakan untuk navigasi robot otonom di lingkungan *indoor* multi lantai pasca gempa bumi.

5.2 Saran

Beberapa saran untuk pengembangan lebih lanjut dari sistem navigasi ini adalah sebagai berikut:

1. **Pengujian di robot fisik**

Pengujian di lingkungan simulasi memberikan gambaran awal mengenai performa sistem navigasi, namun pengujian di robot fisik di lingkungan nyata akan memberikan informasi

lebih mendalam mengenai tantangan yang mungkin muncul, seperti gangguan sensor, kondisi permukaan yang tidak terduga, dan interaksi dengan manusia atau objek lain.

2. Fusion ICP dengan IMU

Lonjakan error ICP di zona transisi akibat minimnya fitur LiDAR dapat dikompensasi dengan sensor IMU yang menyediakan data orientasi dan percepatan pada frekuensi tinggi, sehingga menjaga akurasi lokalisasi secara konsisten di seluruh region.

3. Integrasi dengan Depth Camera

Keterbatasan LiDAR 2D dalam mendeteksi obstacle vertikal dapat dilengkapi dengan data visual dari kamera atau *depth sensor* untuk membangun representasi hibrida 3D-2D, yang memungkinkan robot memahami struktur lingkungan secara lebih utuh tanpa meninggalkan efisiensi navigasi berbasis *occupancy grid*.

4. Pengolahan peta secara otomatis berbasis *point cloud*

Hubungan antar *region* dapat dideteksi secara otomatis menggunakan persamaan *point cloud*. Dengan memanfaatkan algoritma *point cloud registration*, sistem dapat mengidentifikasi korespondensi spasial antar *region* secara langsung dari data *point cloud* 3D, termasuk menghitung matriks transformasi dan orientasi relatif tanpa memerlukan anotasi manual.

DAFTAR PUSTAKA

- Damle, V. P., & Susan, S. (2022). Dynamic algorithm for path planning using a-star with distance constraint. *2022 2nd International Conference on Intelligent Technologies (CONIT)*, 1–5.
- Hu, B., Cui, M., & Cao, Z. (2022). A slope-adaptive navigation approach for ground mobile robots. *2022 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, 610–615.
- Jung, D., Kim, C., Cho, J.-K., & Kim, S.-W. (2024). Munes: Multifloor navigation including elevators and stairs. *arXiv preprint arXiv:2402.04535*.
- Kim, E.-H., Bae, S.-H., & Kuc, T.-Y. (2020). Mobile service robot multi-floor navigation using visual detection and recognition of elevator features (iccas 2020). *2020 20th International Conference on Control, Automation and Systems (ICCAS)*, 982–985.
- Kim, T., Kang, G., Lee, D., & Shim, D. H. (2024). Development of an indoor delivery mobile robot for a multi-floor environment. *IEEE Access*.
- Kusumoputro, B., Purnomo, M. H., Rochardjo, H. S. B., Prabowo, G., Purwanto, D., Pitowarno, E., Mozef, E., Indrawanto, Mutijarsa, K., & Muis, A. (2023). *Pedoman kontes robot indonesia (kri) pendidikan tinggi tahun 2024*.
- Lee, J. Y., Tokuda, T., & Sato, K. (2023). Autonomous floor navigation control of mobile robot using elevator button recognition with deep learning. *2023 62nd Annual Conference of the Society of Instrument and Control Engineers (SICE)*, 133–138.
- Li, S., Chen, Y., Meng, Y., & Lou, Y. (2021). Autonomous elevator button recognition and operation framework for multi-floor mobile manipulator navigation. *2021 IEEE/ASME International Conference on Advanced Intelligent Mechatronics (AIM)*, 383–388.
- Lin, T.-H., Huang, J.-T., & Putranto, A. (2022). Integrated smart robot with earthquake early warning system for automated inspection and emergency response. *Natural hazards*, 110(1), 765–786.
- Lynch, K. M., & Park, F. C. (2017). *Modern robotics*. Cambridge University Press.
- Macenski, S., Moore, T., Lu, D. V., Merzlyakov, A., & Ferguson, M. (2023). From the desks of ros maintainers: A survey of modern & capable mobile robotics algorithms in the robot operating system 2. *Robotics and Autonomous Systems*, 168, 104493.
- Palacín, J., Bitriá, R., Rubies, E., & Clotet, E. (2023). A procedure for taking a remotely controlled elevator with an autonomous mobile robot based on 2d lidar. *Sensors*, 23(13), 6089.
- Palacín, J., Rubies, E., Bitriá, R., & Clotet, E. (2023). Path planning of a mobile delivery robot operating in a multi-story building based on a predefined navigation tree. *Sensors*, 23(21), 8795.
- Sabtaji, A. (2020). Statistik kejadian gempa bumi tektonik tiap provinsi di wilayah indonesia selama 11 tahun pengamatan (2009-2019). *Buletin Meteorologi, Klimatologi, dan Geofisika*, 1(7), 31–46.
- Shenzhen. (2024). Ydlidar t-mini plus data sheet.
- Sobreira, H., Costa, C. M., Sousa, I., Rocha, L., Lima, J., Farias, P., Costa, P., & Moreira, A. P. (2019). Map-matching algorithms for robot self-localization: A comparison be-

- tween perfect match, iterative closest point and normal distributions transform. *Journal of Intelligent & Robotic Systems*, 93, 533–546.
- Thrun, S. (2002). Probabilistic robotics. *Communications of the ACM*, 45(3), 52–57.
- Wei, Z., Wang, S., Chen, K., & Wang, F. (2025). Ros-based navigation and obstacle avoidance: A study of architectures, methods, and trends. *Sensors*, 25(14). <https://doi.org/10.3390/s25144306>
- Xie, H., & Zhou, Q. (2023). Autonomous traversing across floors based on vision for reconfigurable tracked robot. *2023 8th International Conference on Control, Robotics and Cybernetics (CRC)*, 14–19.
- Yao, Q., Tian, Y., Wang, Q., & Wang, S. (2020). Control strategies on path tracking for autonomous vehicle: State of the art and future challenges. *IEEE Access*, 8, 161211–161222.
- Zhang, Y., Li, Y., Zhang, H., Wang, Y., Wang, Z., Ye, Y., Yue, Y., Guo, N., Gao, W., Chen, H., et al. (2023). Earthshaker: A mobile rescue robot for emergencies and disasters through teleoperation and autonomous navigation. *JUSTC*, 53(1), 4–1.
- Zhu, K., Zhan, J., Chen, S., Nan, Z., Zhang, T., Zhu, D., & Zheng, N. (2020). Atv navigation in complex and unstructured environment containing stairs. *2020 IEEE Intelligent Vehicles Symposium (IV)*, 817–824.

LAMPIRAN

[Halaman ini sengaja dikosongkan]

	DEPARTEMEN TEKNIK ELEKTRO FAKULTAS TEKNOLOGI ELEKTRO DAN INFORMATIKA CERDAS INSTITUT TEKNOLOGI SEPULUH NOPEMBER	Nama Dokumen	:	FORM PERNYATAAN PENGGUNAAN AI
		No. Dokumen	:	001/ST_AI/DTE/V/2025
		Halaman	:	1 dari 2

PERNYATAAN KODE ETIK PENGGUNAAN GENERATIVE AI

Code of Conduct Statement For Generative AI or AI-Assisted Usage

Saya yang bertanda tangan di bawah ini:

I the undersigned:

Nama Lengkap / *Full name* : Bagas Surya Wirawan
 NRP / *Student ID* : 5022221026
 Program Studi / *Study Program* : Teknik Elektro / Electrical Engineering
 Judul / *Title* : Navigasi Robot Berbasis Layered Costmap Untuk Pergerakan Antar Lantai Pada Lingkungan Indoor Pasca Gempa Bumi / Robot Navigation Using Layered Costmap For Inter-Floor Movement In Post-Earthquake Indoor Environments
 Pembimbing / *Supervisors* : Dr. Ir. Djoko Purwanto, M.Eng.
 Fajar Budiman, S.T., M.Sc.

Menyatakan dalam pengerjaan Tugas Akhir/Tesis/Disertasi* dengan judul tersebut di atas:

State that during the finalization of my final project/thesis/dissertation with the title above:*

No.	Pernyataan / <i>Statement</i>	Checklist (☒)
1	Hanya menggunakan Generative AI sebagai alat bantu untuk memperbaiki tata bahasa pada Tugas Akhir / Thesis / Disertasi saya. Ini tidak digunakan untuk membuat keseluruhan isi pekerjaan saya. <i>Use Generative AI only as a tool to improve the readability or language of the text in my Final Project / Thesis / Dissertation*. It is not used to generate a complete text of my work.</i>	✓
2	Telah memeriksa dan/atau memperbaiki seluruh bagian dari pekerjaan saya yang dibantu oleh Generative AI agar sesuai dengan baku mutu penulisan karya ilmiah <i>Has checked and/or refined all parts of my work that is assisted by Generative AI in such a way that it complies with the standard of academic writing</i>	✓
3	Tidak menggunakan Generative AI untuk pembuatan data primer, grafik dan/atau tabel pada pekerjaan saya. <i>Do not use Generative AI to generate primary data, figure and/or table in my works.</i>	✓

	DEPARTEMEN TEKNIK ELEKTRO FAKULTAS TEKNOLOGI ELEKTRO DAN INFORMATIKA CERDAS INSTITUT TEKNOLOGI SEPULUH NOPEMBER	Nama Dokumen	:	FORM PERNYATAAN PENGGUNAAN AI
		No. Dokumen	:	001/ST_AI/DTE/V/2025
		Halaman	:	2 dari 2

No.	Pernyataan / Statement	Checklist (☒)
4	Penulis telah memberikan atribusi/pengakuan terhadap alat AI yang digunakan, secara rinci pada suatu bagian pada lampiran. <i>The author has acknowledged the use of Generative AI in any part of the work in the specific appendix page.</i>	✓
5	Buku tugas akhir / Tesis / Disertasi* telah mematuhi Baku Mutu Akademik ITS terkait keaslian karya dan etika penulisan. <i>The final project/thesis/dissertation* has complied with the Academic Standard of ITS related to the originality of scientific work and scientific writing code of conduct</i>	✓
6	Penulis memastikan tidak ada plagiarisme, termasuk yang berasal dari penggunaan AI. <i>The authors has ensured that there is plagiarism issue in the work, including any part that is generated by AI.</i>	✓

Catatan/note: * Pilih salah satu yang sesuai

Tempat/ Place	:	Surabaya
Tanggal/ Date	:	Kamis, 18 Juni 2026
Nama Lengkap/ Full Name	:	Bagas Surya Wirawan
Tanda tangan/ Signature	:	

BIOGRAFI PENULIS



Penulis ini bernama Bagas Surya Wirawan, lahir di Jakarta hari Sabtu tanggal 2 April 2004. Penulis ini merupakan mahasiswa program studi Teknik Elektro di Institut Teknologi Sepuluh Nopember (ITS) dan bidang studi Elektronika. Penulis memiliki minat di bidang robotika dan sistem navigasi, yang mendorong penulis memilih judul ini. Selama masa perkuliahan penulis aktif mengikuti kegiatan luar kuliah, salah satunya di Tim Robotika ITS dimana penulis merupakan salah satu anggota Tim Abinara-1. Kedepannya, penulis berharap bisa menerapkan ilmu robotika, baik dalam dunia kerja maupun program magister.

[Halaman ini sengaja dikosongkan]